Grab UC4 Cognigy TDD: Benchmark Scope Edition

Oct 6, 2026 · @Aryo Nugraha

1. Document Control and Purpose

This edition narrows the Grab UC4 Cognigy build to what NiCE's analysis asks for first: the 15 benchmark test cases, TC-001 to TC-010 (GrabFood) and TC-025 to TC-029 (GrabMart). It also names the capabilities NiCE marks as worth building now even though they are not tested first, and it defers everything else.

The full design stays in Grab UC4 Merchant Voicebot: Cognigy TDD v0.3. This edition is self contained: it carries every tool this phase needs with its schema, logic and code, decides per feature what is built and accepted in this phase, and records where this phase departs from v0.3.

1.1 Revision history

Version	Date	Author	Summary of change
1.0	2026-10-06	Amansob	First benchmark scope edition, based on TDD v0.3 and the NiCE analysis of Grab's documents.

1.2 Source documents

Ref	Document	Role in this edition
SRC-1	Grab Pilot Test Cases, 30 Sep 2026 (UC4 Test Cases ENG)	The 15 benchmark cases (column F = Yes) are the acceptance criteria for this phase.
SRC-2	Grab Pilot Test Cases, Scenario Flow	Node scripts in English and Malay.
SRC-3	Grab Merchants Voicebot Use Case 4 (Wiz AI V4)	Behavioural reference.

Ref	Document	Role in this edition
SRC-4	NiCE: Analysis of Documents from Grab	Defines the benchmark set and the "worth building, not tested first" list that sets this edition's scope.
TDD v0.3	Grab UC4 Merchant Voicebot: Cognigy TDD v0.3	Full design: architecture, context model, tool specifications and code.

1.3 How to use this edition

Developers build from Section 6, which holds every tool, its schema, its logic and its code, plus the shared configuration and state. Section 5 says which features belong to Band 1 and which to Band 2. Testers accept the phase on Band 1 only (Section 8). No other document is needed to build this phase; TDD v0.3 remains the reference for the full design beyond it.

2. Scope Bands

Every capability in TDD v0.3 falls into one of three bands. Band 1 is what the phase is accepted on. Band 2 is built in the same phase, because it is cheap once Band 1 exists and NiCE expects it, but it is tested after the benchmark cases pass. Band 3 is not built in this phase.

Band	Definition	Test cases	Acceptance
Band 1: Benchmark	Everything the 15 benchmark cases exercise, in English, with Manglish understood	TC-001 to TC-010, TC-025 to TC-029	Phase acceptance. Each case passes 3 runs over WebRTC.
Band 2: Build now, test later	NiCE's "worth building, not tested first" list	TC-011 to TC-024, TC-030 to TC-045	Tested after Band 1 passes. Defects logged, not blocking the benchmark.
Band 3: Deferred	Not needed by any Band 1 or Band 2 case	None	Not built in this phase.

2.1 Band 1: Benchmark

- Order age and threshold for both LOBs, with the timeline and photo reminder (TC-001, TC-004, TC-025, TC-028).
- First follow up answer and a declined offer of guidance (TC-002, TC-026).
- Guidance preferences: platform, style, loudspeaker (TC-003, TC-005, TC-027, TC-029).
- GrabFood status check in the app, with the History fallback, and GrabMart status check on the portal with the partial compensation line (TC-003, TC-005, TC-007, TC-027, TC-029).
- Request for Compensation in the app and on the portal (search route), including the app to portal fallback (TC-003, TC-005 to TC-009, TC-027, TC-029).
- Form not visible: scroll, Owner or Manager reminder, retry, Final Educate or an offer of a person (TC-006, TC-009).
- Stuck on a step: repeat, then recover (TC-007).
- Out of scope request: consent, then mock transfer (TC-010).
- Mock transfer and Closing A. Manglish understood in an English conversation.

2.2 Band 2: Build now, test later

Capability (from SRC-4)	Test cases	Note
Safety escalation, upfront and mid flow	TC-011, TC-012, TC-034, TC-035	escalate_safety tool
Portal find method ladder	TC-013 to TC-016, TC-036 to TC-039	Routes 2 and 3. All three failing ends in a soft close, not a transfer (E-03).
View Cancelled Order journey	TC-017, TC-040	app_view_order and portal_view_order segments
Hard stuck, then transfer	TC-018, TC-041	Full stuck ladder
Unclear or unparseable date	TC-019, TC-042	Proceeds silently on the beyond path, never announcing an assumption
Dissatisfaction, then explicit request for a person	TC-020, TC-043	

Capability (from SRC-4)	Test cases	Note
Two separate orders in one call	TC-021, TC-044	Per order caps, preferences carried over
Mid call date correction	TC-022, TC-045	Silent recalculation and re-route
Malay language request	TC-023, TC-024	Tested as written by SRC-4 (E-04)
Full Bahasa Melayu conversation	Malay run of the benchmark cases	Malay language pack (6.5) and switching (6.8), E-04
GrabMart mirrors of role, stuck and out of scope paths	TC-030 to TC-033	Same tools as TC-006, TC-007, TC-009, TC-010

2.3 Band 3: Deferred

- search_knowledge and the Tier 3 knowledge store (E-02).
- Live transfer to CXone, SIP headers and ACW delivery to CXone. The transfer stays a mock.
- Telephony. The phase runs on Cognigy's WebRTC voice client.

3. Requirements from the Benchmark Cases

These requirements are read from the 15 benchmark transcripts and flow paths in SRC-1, checked against SRC-3. Every Band 1 feature in Section 5 traces to at least one of them. SRC-1 accepts any wording that shares the same objective, except the lines returned with verbatim true (Section 6).

ID	Requirement	Benchmark cases
R-01	Turn the merchant's date answer (days, weekday, calendar date, Malay or Manglish words) into a within or beyond verdict using the weekday tables for each LOB.	TC-001 to TC-005, TC-025 to TC-029
R-02	Within threshold: the LOB timeline, the photo reminder (verbatim) and the anything else question in one turn. No satisfaction question.	TC-001, TC-025

ID	Requirement	Benchmark cases
R-03	Beyond threshold with no stated intent: ask what they would like to know, then give the timeline and offer to show the compensation status.	TC-004, TC-028
R-04	First follow up after the timeline: the further inquiry script with an offer to guide. A no leads to the anything else question and Closing A.	TC-002, TC-026
R-05	Guidance preferences one per turn: platform, then style, then loudspeaker.	TC-003, TC-005, TC-027, TC-029
R-06	Status and dispute both wanted: status check first, then the submission.	TC-003, TC-005, TC-027, TC-029
R-07	GrabFood status in the app: Orders tile, Cancelled tab, read the status. No status means the next step is a Request for Compensation.	TC-003, TC-005, TC-006, TC-008
R-08	Order not in the Cancelled tab (older than this month): History tab, Date selector, Confirm.	TC-007
R-09	GrabMart status on the portal only (Finance, Transactions, period and store, Category), with a short redirect when the merchant chose the app, and the partial compensation line said verbatim.	TC-027, TC-029
R-10	App submission: Request for compensation link, Report a payment dispute form, fill with the LOB evidence, submit, then the after submission line.	TC-003, TC-005 to TC-007
R-11	Request for compensation link missing in the app: offer the portal, then Help Centre and the Search Bar route.	TC-008
R-12	Form not visible: scroll first, then the Owner or Manager reminder with a retry offer. Retry works: fill and submit. Retry declined: Final Educate. Retry fails: offer a person, then mock transfer.	TC-006, TC-009

ID	Requirement	Benchmark cases
R-13	Stuck on a step: repeat that step once with where it is on screen. The merchant recovers and the walkthrough continues. No early offer of a person.	TC-007
R-14	Out of scope request: say the bot only handles cancelled orders, ask for consent, then mock transfer.	TC-010
R-15	End with Closing A after the merchant says there is nothing else.	All
R-16	Evidence wording by LOB: the prepared food for GrabFood, the packed items for GrabMart.	TC-003, TC-005, TC-027, TC-029
R-17	Understand Manglish in an English conversation without switching language.	All, on real speech

4. Decisions for This Edition

The decisions below apply to this phase. The design rules carried over from TDD v0.3 are built into Section 6. They include the weekday threshold tables, the role retry path, the GrabMart portal only status check, the three content tiers, state in context.shortTermMemory.uc4, and Code nodes without return statements.

ID	Decision	Why	Changes from v0.3
E-01	Build in benchmark slices: each slice delivers a group of benchmark cases end to end before Band 2 work starts (Section 7).	Gets the 15 accepted cases working early, so testing starts in week 2 instead of after feature complete.	Build order only

ID	Decision	Why	Changes from v0.3
E-02	search_knowledge is deferred. knowledgeEnabled = false, the tool is not added to the AI Agent, and no knowledge content is loaded.	No Band 1 or Band 2 case needs it. Benchmark follow ups are covered by fixed and guided answers. Removes a content dependency and a source of ungrounded answers.	D-09, D-17 deferred. Eight tools instead of nine.
E-03	All three portal routes failing ends in a soft close, not an offer of a person. portalExhaustedAction = soft_close.	SRC-4 states this for TC-016 and TC-039. The flag can flip back if Grab confirms the transcript behaviour.	Replaces D-04 for this phase
E-04	Band 1 runs in English and understands Manglish: both Set Session Config nodes recognise English and Malay together. Full Bahasa Melayu conversations (language packs in 6.5, switching in 6.8) are built in Band 2. TC-023 and TC-024 are tested as SRC-4 describes them (merchant accepts English, or declines and is transferred) with languageSwitch = ask_then_transfer for those runs.	The benchmark is English. Manglish appears on real calls, so it is needed from day one. Malay conversation stays in the build, but outside the acceptance gate.	D-12 production setting pending Grab (OI-09)
E-05	The phase runs on Cognigy's WebRTC voice client, and the transfer is a mock: transfer message, ACW written to the log, then hang up.	No telephony in the pilot. Grab confirms in writing that a mock transfer passes TC-009 and TC-010.	CXone handover deferred; mock transfer in 6.12

ID	Decision	Why	Changes from v0.3
E-06	Band 1 uses the first rung of the stuck ladder only (repeat). The full ladder (repeat, repeat, clarify, clarify, offer) is built in Band 2.	TC-007 recovers after one repeat. TC-018 and TC-041 test the full ladder.	None to the code; testing split only

AI Agent instructions for this phase are in 6.15. They leave out search_knowledge: a compensation question the tools do not cover gets "I don't have that detail on hand" and the anything else question.

5. Tool and Component Scope

The AI Agent carries eight tools in this phase. Each tool is specified in full in Section 6. This section says which of its features belong to Band 1 and which to Band 2. Band 1 features are built and tested first; Band 2 features follow in the same phase.

5.1 Tool scope matrix

Tool	Band 1 (benchmark)	Band 2 (build now, test later)	Section
resolve_order_age	days_ago, weekday and calendar date; both threshold tables; intents timeline, status, submit, status_and_submit; timeline and photo reminder	Unclear date cap and silent beyond (TC-019, TC-042); is_correction (TC-022, TC-045); is_new_order with preferences carried over (TC-021, TC-044)	6.7

Tool	Band 1 (benchmark)	Band 2 (build now, test later)	Section
get_approved_answer	further_inquiry; out_of_scope with an offer of a person	dissatisfied and human_request (TC-020, TC-043); language_request (TC-023, TC-024 and Malay conversation); evidence_question, photo_missing, not_prepared, payout_unclear, other_brand	6.8
set_guidance_preference	Full tool: platform, style, loudspeaker, two asks cap, assumptions	None	6.9
start_walkthrough	Journeys check_status, submit_request and status_then_submit; GrabMart portal redirect and partial compensation line	Journey view_order (TC-017, TC-040)	6.10

Tool	Band 1 (benchmark)	Band 2 (build now, test later)	Section
advance_walkthrough	confirmed; failed with the first rung (repeat); status_blank and status_compensated; order_not_visible with the History fallback; link_not_found with the portal offer; route_not_found on route 1 only; form_not_visible, scrolled_no_form, form_visible; role retry yes and no; submitted; needs_time	Full stuck ladder to an offer (TC-018, TC-041); routes 2 and 3 and the soft close (TC-013 to TC-016, TC-036 to TC-039); platform_switch; skip_to_step	6.11
transfer_to_live_agent	yes and no, mock transfer (TC-009, TC-010)	Language reason and its exit (TC-024)	6.12
escalate_safety	None	Full tool (TC-011, TC-012, TC-034, TC-035)	6.13
end_call	closing_a	disengaged, could_not_resolve, language_not_supported	6.14
search_knowledge	Not built (E-02)	Not built	Deferred

The decision chain in advance_walkthrough is written in full in Band 1, because its branches share helpers. Band 2 here means those branches are tested later, not that the code is left out.

5.2 Other components

Component	Band 1	Band 2	Section
Start up flow: Code: Config, language packs, Code: Initiate, Once, greeting	Built in full		6.3, 6.5
Step catalogue	Loaded in full as data. Segments tested in Band 1: food_app_status, food_app_history, portal_status, app_submit, portal_submit (route 1)	app_view_order, portal_view_order, portal routes 2 and 3	6.6
Set Session Config	English node with Malay as an alternate STT language, so Manglish is transcribed	Malay node and the language If	6.3
Language packs	English pack. The Malay pack falls back to English until it is loaded.	Malay pack loaded after Grab approval (OI-21)	6.5
Silence handling	Built in full		6.3
Mock transfer and ACW to log	Built		6.12

5.3 Phase overrides in config

This phase's settings are built directly into Code: Config (6.5), so there is no separate override node:

Setting	Value	Decision
flags.portalExhaustedAction	"soft_close"	E-03
flags.knowledgeEnabled	false	E-02

Setting	Value	Decision
flags.languageSwitch	"ask_then_transfer" (set to "switch" for Malay conversation tests in Band 2)	E-04
transferMode	"mock"	E-05
content.ms	Falls back to the English pack until the Malay pack is loaded	E-04

6. Tool Catalogue and Implementation

This section is the build specification. It holds everything a developer needs: the eight tools with their descriptions, JSON schemas, logic and Code node code, the shared configuration, the session state and the AI Agent instructions. Section 5 says which features are Band 1 and which are Band 2.

6.1 Catalogue

The AI Agent node has eight tools. Six always return a result to the agent through actions.output. escalate_safety and end_call are terminal, and transfer_to_live_agent is terminal when the merchant says yes. Each tool's Condition field is a guard: Cognigy hides the tool from the LLM while the guard is false.

Tool	Purpose	Condition (guard)	Returns or terminal	Band
resolve_order_age	Within or beyond threshold, date question cap, corrections, second order	context.shortTermMemory.uc4.closingSaid !== true	Returns	1 and 2
get_approved_answer	Fixed and guided answers, language requests, every offer of a person that is not a walkthrough failure	context.shortTermMemory.uc4.closingSaid !== true	Returns	1 and 2
set_guidance_preference	Platform, style and loudspeaker, one per turn	context.shortTermMemory.uc4.orders.length > 0	Returns	1
start_walkthrough	Pick the step segment for LOB, journey and platform, deliver the first step	context.shortTermMemory.uc4.prefs.platform !== "" && context.shortTermMemory.uc4.prefs.style !== ""	Returns	1 and 2

Tool	Purpose	Condition (guard)	Returns or terminal	Band
advance_walkthrough	Move the walkthrough on each merchant reply: stuck ladder, route ladder, role path	context.shortTermMemory.uc4.walk.status === "active"	Returns	1 and 2
transfer_to_live_agent	Act on the merchant's yes or no to an offer of a person (mock transfer)	context.shortTermMemory.uc4.transfer.offered === true	Terminal on yes, returns on no	1
escalate_safety	Immediate mock transfer for a live emergency, no consent	(none, always available)	Terminal	2
end_call	Play the exit message and hang up	context.shortTermMemory.uc4.closingSaid !== true	Terminal	1 and 2

6.2 Result contract

Every returning tool sends the same shape, so the AI Agent instructions say "follow next_action" once.

```
{
  "success": true,
  "next_action": "deliver_step",
  "say": "Please open the GrabMerchant app. Can you see the home screen?",
  "verbatim": true,
  "guide": "",
  "step_id": "open_app"
}
```

say carries Tier 1 text (verbatim true, said exactly) or text the agent may rephrase (verbatim false). guide carries a Tier 2 instruction: the agent writes its own reply that does exactly what guide says and keeps every fact in it. Tier 1 text lives in config (6.5, 6.6). Tier 2 guides are written in English for the LLM, which replies in the conversation language.

next_action	What the AI Agent does
deliver_say	Say the say text (exactly, when verbatim is true) and wait.
deliver_guided	Write one short reply that does what guide says, keeping every fact, and wait.

next_action	What the AI Agent does
deliver_step	Say the step exactly. It always ends with a check back.
ask_date	Ask when the order was cancelled, using say.
ask_open_question	Ask what they would like to know about that order. Never mention the date or any assumption.
ask_preference	Ask exactly the one preference question in say.
collect_preferences	Say the say text if any, then call set_guidance_preference.
start_walkthrough	Then call start_walkthrough with the journey in guide; mention any assumption guide asks for.
offer_transfer	Ask the consent question in say. The answer goes to transfer_to_live_agent.
hold	Acknowledge briefly as guide says and wait. No new question.
steer_back	Do not repeat earlier content. Follow guide if filled, else re-ask the current question or ask if there is anything else.
end_call	Call end_call with the exit_type in guide.
terminal	Nothing to say. The tool path has ended the turn.
apologise_and_retry_once	A tool error. Apologise briefly and ask the merchant to repeat.

6.3 Conventions, start up flow and voice settings

Code node conventions (cognigy-code-patterns)

- Read arguments from input.aiAgent.toolArgs, guarded with || { }.
- Guard shortTermMemory before use: if (!context.shortTermMemory) { context.shortTermMemory = {}; }. Write only to context.shortTermMemory.uc4.
- No return keyword anywhere in a Code node, including inside helpers. Logic is an if/else chain that assigns a variable named out. Helpers are expression arrow functions, or block functions that write to out.

- Every returning tool ends with actions.output("", out). The empty first argument keeps it silent; out goes to the AI Agent as the tool result. Keep a copy in input.uc4Result.
- Never write null into context. Use "", -1 or false. Do not use actions.addToContext on shortTermMemory.
- Code nodes cannot share functions, so small helpers (res, deliver) are repeated in each tool.

Tool path pattern. Returning tool: Code node, then Resolve Tool Action. Terminal tool: Code node, Code: Build ACW, Say node with a fixed config message, then Hang Up (or the mock transfer path). The LLM never speaks after a terminal message. Confirm in slice 1 that the actions.output payload reaches the agent in the project's Cognigy version (OI-19); if not, set the Resolve Tool Action answer to {{JSON.stringify(input.uc4Result)}}.

Start up flow (once per call)

#	Node	Purpose
1	Start	Flow entry.
2	Once	First branch runs nodes 3 to 9. Afterwards branch goes to node 10.
3	Code: Config	Loads context.uc4Cfg: flags, thresholds, English Tier 1 text, guides, messages (6.5).
4	Code: Config (steps)	Loads the step catalogue and walkthrough text (6.6).
5	Code: Config (language packs)	Builds context.uc4Cfg.content for en and ms (6.5).
6	Code: Initiate	Reads lob and lang from the WebRTC test page, builds context.shortTermMemory.uc4, applies the language pack (6.5).
7	If: LOB valid	context.shortTermMemory.uc4Valid false: Say a short apology and Hang Up.
8	Set Session Config	Voice settings below. Band 2 adds a Malay node behind an If on uc4.lang.
9	Say: Greeting	{{context.uc4Cfg.messages.greeting}}. Ends the execution and waits for the merchant.
10	AI Agent	Handles every merchant turn with the eight tools.

Set Session Config (English node, Band 1)

Parameter	Value
STT	English (Malaysia where supported, else Singapore) primary, Malay (ms-MY) alternate, so Manglish is transcribed
Phrase hints	GrabMerchant, GrabFood, GrabMart, app, portal, cancelled, compensation, dispute, Request for Compensation, Report a payment dispute, Help Centre, Orders tile, History, Owner, Manager, payout, loudspeaker, pampasan, batal, semalam, kelmarin, dah, tak nampak, takde

Parameter	Value
TTS	English voice approved by Grab
Barge-in	On, speech triggered, minimum two words
No input timeout	7 seconds by default, 20 seconds while a walkthrough is active

Silence handling. A user input timeout is handled before the AI Agent node, so silence never counts toward the stuck ladder. First timeout: "Take your time, I'm still here" (during a walkthrough) or repeat the last question. Second: "Are you still there?" Third: end_call path with exit_type disengaged.

```
// Code: No Input Counter. Runs when the input is a user input timeout. No
return statements.
if (!context.shortTermMemory) {
  context.shortTermMemory = {};
}
const u = context.shortTermMemory.uc4;
u.noInput = (u.noInput || 0) + 1;
if (u.noInput >= 3) {
  input.uc4NoInputAction = "end_disengaged";
} else if (u.noInput === 2) {
  input.uc4NoInputAction = "check_presence";
} else {
  input.uc4NoInputAction = "reassure";
}
// Reset u.noInput = 0 at the top of every real merchant input.
```

6.4 Session state: context.shortTermMemory.uc4

All session state lives under context.shortTermMemory.uc4, and all configuration under context.uc4Cfg. Tools are the only writers of the uc4 state. The AI Agent reads state only through tool results, never by inferring it from the transcript.

Field	Default	Written by	Description
lob	from the test page	Code: Initiate	"GrabFood" or "GrabMart". Never changes and is never asked.
lang, langName	"en", "English"	Code: Initiate, get_approved_answer	Conversation language: "en" or "ms". Starts from the test page and then follows the merchant (Band 2).
callId	from the test page	Code: Initiate	Session ID for test evidence and the ACW log.
today	MYT date	Code: Initiate	{ iso, weekday } in Asia/Kuala_Lumpur time, or cfg.testToday outside production.
orders	[ ]	resolve_order_age	One object per order raised in the call.
activeOrder	""	resolve_order_age	Ref of the order in hand: "A", then "B".
prefs	see below	set_guidance_preference	Platform, style, loudspeaker. Kept for the whole call.
walk	see below	start_walkthrough, advance_walkthrough	The current walkthrough.
transfer	{ offered: false, reason: "", declined: false, executed: false, unclearOnce: false }	get_approved_answer, advance_walkthrough, transfer_to_live_agent	Offer and decline tracking. A declined offer is not repeated unless the merchant asks for a person.

Field	Default	Written by	Description
language	{ switches: 0, otherAsks: 0 }	get_approved_answer	Language switches and requests for unsupported languages.
dissatisfaction	{ attempts: 0 }	get_approved_answer	One real retry before an offer.
outcome	{ journeys: [ ], result: "" }	All tools	Summary for the ACW log.
safetySummary, noInput, closingSaid	"", 0, false	escalate_safety, silence counter, end_call	Once closingSaid is true, nothing more is spoken.

Order object (uc4.orders[ ])

Field	Default	Description
ref	"A"	"A" for the first order, "B" for the next.
ageDays	-1	Days since cancellation. -1 means unknown.
dateSource	""	days, weekday, date or assumed.
threshold	""	within or beyond. Set only by resolve_order_age.
dateAskCount	0	Bot turns that asked for the date. Capped at 2.
educateDelivered, followupAnswered	false	The timeline and the first follow up were given.
submitted, closedAfterRole	false	A request was submitted, or the order was closed with Final Educate.

Preferences (uc4.prefs): platform ("app" or "portal"), style ("one_by_one" or "all_at_once"), loudspeaker ("yes" or "no"), asks (count per question, max 2, loudspeaker 1), assumed (list of preferences assumed).

Walkthrough (uc4.walk)

Field	Default	Description
journey, pendingJourney	""	view_order, check_status, submit_request or status_then_submit. pendingJourney is set before preferences are complete.
platform, segment	""	Active platform and step segment (6.6).
stepIndex, stuckCount	0	Current step and failure reports on it.
routeIndex	0	Portal Help Centre route: 0 search bar, 1 common problems, 2 browse by services.
scrollAsked, roleAttempts, roleState	false, 0, ""	Role path state.
awaiting	""	The yes or no question pending: submit_choice, cross_build or role_retry.
then	""	Segment to chain into after this one (app_submit or submit).
crossBuildOffered, portalOpened, orderDetailsOpen, disclaimerSaid	false	Fallback offered, portal already open, order details already open, partial compensation line already said.
status	""	active, completed or handed_over.

6.5 Configuration and initiation

All business values live in Code: Config, so a Grab decision is a config change, not a code change. Tier 1 text is said exactly. Tier 2 guides are instructions to the LLM, which phrases the reply. The phase settings from Section 5.3 are already set here.

Code: Config

```
// Code: Config. Runs first. All business values in one place. No return
statements.
// fixed, messages and evidence below are the ENGLISH pack. The Malay pack
has the same keys (Band 2).
context.uc4Cfg = {
  timezone: "Asia/Kuala_Lumpur",
  testToday: "", // non production only, for example { iso: "2026-10-07",
weekday: "Wed" }
  transferMode: "mock", // E-05
  flags: {
    maxDateAsks: 2,
    maxPrefAsks: 2,
    stuckLadder: ["repeat", "repeat", "clarify", "clarify",
"offer_transfer"], // Band 1 tests the first rung only
    portalExhaustedAction: "soft_close",       // E-03
    roleRetryEnabled: true,
    roleMaxAttempts: 2,
    knowledgeEnabled: false,                   // E-02
    languageSwitch: "ask_then_transfer"        // E-04; "switch" for Malay
conversation tests
  },
  // Within threshold when ageDays <= value for today's weekday
  thresholdDays: {
    GrabFood: { Mon: 4, Tue: 4, Wed: 2, Thu: 2, Fri: 2, Sat: 3, Sun: 4 },
    GrabMart: { Mon: 7, Tue: 7, Wed: 7, Thu: 7, Fri: 7, Sat: 8, Sun: 9 }
  },
  // A bare weekday name: within only if listed for today (GrabFood).
GrabMart: any weekday in the past week.
  weekdayWithin: {
    GrabFood: { Mon: ["Sun", "Sat", "Fri", "Thu"], Tue: ["Mon", "Sun", "Sat",
"Fri"], Wed: ["Tue", "Mon"],
                Thu: ["Wed", "Tue"], Fri: ["Thu", "Wed"], Sat: ["Fri", "Thu",
"Wed"], Sun: ["Sat", "Fri", "Thu", "Wed"] },
    GrabMart: "any"
  },
  evidence: { GrabFood: "a photo of the prepared food", GrabMart: "a photo of
the packed items" },

  // Tier 1: fixed, said exactly (English pack)
  fixed: {
    GrabFood: {
      educate: "Compensation for a cancelled order is usually approved within
one to two business days after the order was made, and it is then paid in
```

```
your next payout cycle, the same cycle as your regular order earnings,
subject to Grab's cancellation policy and guidelines.",
      photoReminder: "Please take a photo of the prepared food as evidence,
in case you would like to submit a request for our support team to review
further after it has been more than two business days."
    },
    GrabMart: {
      educate: "Compensation for a cancelled order is usually approved within
three to five business days after the order was made, and it is then paid in
your next payout cycle, the same cycle as your regular order earnings,
subject to Grab's cancellation policy and guidelines.",
      photoReminder: "Please take a photo of the order as evidence, in case
you would like to submit a request for our support team to review further
after it has been more than five business days.",
      partialDisclaimer: "Please note that compensation will be partial,
based on the assessment of the perishable items."
    },
    furtherInquiry: "If you're eligible for compensation, you'll see the
Request for Compensation section within that cancelled order in your
GrabMerchant app. If you can't see it, you should submit a request for our
support team to review further. Would you like to be guided to your cancelled
order?",
    orderVisible: "Now you can view the details of your order, and you might
also see the Request for Compensation section within it. If you can't see it,
you should submit a request for our support team to review further. Would you
like help submitting a Request for Compensation, or is that all you needed?",
    afterSubmission: "Once approved, you may receive the compensation
depending on the review outcome. That's all I can assist you with for now.",
    offerPerson: "Would you like me to connect you with someone who can help
further?",
    outOfScopeLead: "I only handle issues about cancelled orders.",
    otherBrand: "This line only covers {lob} cancelled order compensation.
Would you like to speak to someone who can help with your other order?",
    udr: "Is there anything else I can help you with?",
    phrases: {
      separateOrder: "Sure, let's take that as a separate order.",
      askDate1: "When was the cancelled order made?",
      askDate2: "Could you give me a rough idea, roughly how many days ago it
was, or which day?",
      openQuestion: "Could you explain what you'd like to know?",
      canSubmit: "You can submit a Request for Compensation.",
      offerStatusCheck: "Would you like me to show you how to check the
compensation status for that order?",
```

```
      askPlatform: "Are you on the GrabMerchant app, or the GrabMerchant
portal on a computer?",
      askStyle: "Would you like the steps one by one, or all at once?",
      askLoudspeaker: "Would you like to put me on loudspeaker before we
start?",
      understand: "I understand.",
      sorryNotHelped: "I'm sorry this hasn't helped.",
      sorryStuck: "I'm sorry you're running into that.",
      ownerManagerOnly: "Only an Owner or Manager can submit a request.",
      noneWorked: "I'm sorry, none of those got us to the form.",
      switched: "Sure, let's continue in English.",
      stayInLanguage: "I can only converse in English here. Would you like to
proceed in English?",
      languagesOffered: "I can help in English or Bahasa Melayu. Which would
you prefer?"
    }
  },

  // Tier 2: guided. Written in English for the LLM; it replies in the
conversation language.
  guided: {
    evidence: "Tell them the main evidence is {evidence}. One sentence.",
    photoMissing: "Reassure them first that they can still fill in and submit
the form without the photo. Then suggest they include all the information and
documents they have, clear and complete, so the review is faster. Never make
the photo a condition.",
    notPrepared: "Explain that compensation applies only to orders cancelled
after they were already prepared, so it would not apply in this case. Then
ask if there is anything else you can help with.",
    payoutUnclear: "Ask one short question: is this about the payout for an
order that was cancelled after it was prepared?",
    alreadySubmitted: "Tell them a request has already been submitted for
this order in this call, so the next step is to wait for the outcome. Then
ask if there is anything else.",
    finalEducate: "Ask them to try logging in with an Owner or Manager
account first, and say they are welcome to call back if they still cannot
access the form after that.",
    dissatisfiedFirst: "Acknowledge their frustration in one sentence, then
make one real attempt to help with the current order. Do not offer a
person.",
    humanFirst: "Say you can help with their cancelled order and ask what
they need. Do not offer a person yet.",
    alreadyShared: "Say briefly that you have shared what you can on that,
```

```
then ask if there is anything else.",
    declinedPerson: "Acknowledge the no in a few words, then ask if there is
anything else you can help with.",
    continueOrder: "Acknowledge, and carry on with the {lob} order you were
helping with."
  },

  // Terminal messages: fixed, played by Say nodes, never by the LLM (English
pack)
  messages: {
    greeting: "Hi, I'm your Grab AI Assistant, and I'll be assisting you
today. Please note that this call will be recorded for quality assurance and
training purposes. I understand that you're contacting us about an order that
was cancelled after it had already been prepared. How may I help you today?",
    transfer: "Please hold while I transfer you to a customer service
officer. You might have to repeat some of our conversation. Thank you for
your patience and understanding.",
    safety: "I understand this involves a safety concern. If anyone is hurt,
please call your local emergency number now. Let me connect you to someone
who can help right away.",
    exitClosingA: "Thank you for calling Grab. If there's anything else you
need help with, please contact us again. Have a nice day.",
    exitCouldNotResolve: "I'm sorry we can't help you further with this
today. If anything else comes up, please don't hesitate to call us back.",
    exitDisengaged: "No problem, you're welcome to call back anytime for
further help. Take care.",
    exitLanguage: "I'm sorry, I can only help in English or Bahasa Melayu, so
I'm not able to assist you today. Please feel free to call us back."
  }
};
```

Code: Config (language packs)

Every Tier 1 object (fixed, steps, walkFixed, messages, evidence) exists once per language with the same keys. Tools read the active pack, so they contain no language logic. The Malay pack is pasted from the UC4 Content Pack (Malay sheet) once Grab approves it (OI-21); until then a Malay start falls back to English.

```
// Code: Config (language packs). Runs after Code: Config and Code: Config
(steps). No return statements.
const c = context.uc4Cfg;
const en = { fixed: c.fixed, steps: c.steps, walkFixed: c.walkFixed,
messages: c.messages, evidence: c.evidence };
const ms = null; // Band 2: replace null with the approved Malay pack object
(same keys as en)
c.content = { en: en, ms: ms ? ms : en };
```

Code: Initiate

```
// Code: Initiate. Runs once per call, after the three Code: Config nodes. No
return statements.
// The WebRTC test page passes lob and, optionally, lang as session data.
// For Interaction Panel testing, send input.data = { "lob": "GrabFood",
"lang": "en" }.
if (!context.shortTermMemory) {
  context.shortTermMemory = {};
}
const callData = input.data || {};
const rawLob = String(callData.lob || "").toLowerCase();
const rawLang = String(callData.lang || "en").toLowerCase();

let lob = "";
if (rawLob.indexOf("food") !== -1) {
  lob = "GrabFood";
} else if (rawLob.indexOf("mart") !== -1) {
  lob = "GrabMart";
}
let lang = "en";
if (rawLang.indexOf("ms") === 0 || rawLang.indexOf("bm") === 0 ||
rawLang.indexOf("malay") !== -1 || rawLang.indexOf("melayu") !== -1) {
  lang = "ms";
}

// Today in Malaysia time (UTC+8). testToday overrides it outside production.
const my = new Date(Date.now() + 8 * 3600 * 1000);
const weekdays = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
const realToday = { iso: my.toISOString().slice(0, 10), weekday:
weekdays[my.getUTCDay()] };

context.shortTermMemory.uc4 = {
  lob: lob,
```

```
  lang: lang,
  langName: lang === "ms" ? "Bahasa Melayu" : "English",
  callId: String(callData.sessionId || callData.contactId || ""),
  today: context.uc4Cfg.testToday ? context.uc4Cfg.testToday : realToday,
  orders: [],
  activeOrder: "",
  prefs: { platform: "", style: "", loudspeaker: "", asks: { platform: 0,
style: 0, loudspeaker: 0 }, assumed: [] },
  walk: { journey: "", pendingJourney: "", platform: "", segment: "",
stepIndex: 0, stuckCount: 0, routeIndex: 0,
          scrollAsked: false, roleAttempts: 0, roleState: "", awaiting: "",
then: "", crossBuildOffered: false,
          portalOpened: false, orderDetailsOpen: false, disclaimerSaid:
false, status: "" },
  transfer: { offered: false, reason: "", declined: false, executed: false,
unclearOnce: false },
  language: { switches: 0, otherAsks: 0 },
  dissatisfaction: { attempts: 0 },
  outcome: { journeys: [], result: "" },
  safetySummary: "",
  noInput: 0,
  closingSaid: false
};

// Apply the language pack for the starting language
const pack = context.uc4Cfg.content[lang];
Object.assign(context.uc4Cfg, { fixed: pack.fixed, steps: pack.steps,
walkFixed: pack.walkFixed,
  messages: pack.messages, evidence: pack.evidence });
context.shortTermMemory.uc4Valid = (lob !== "");
```

6.6 Step catalogue

Every screen instruction is held as data, so the walkthrough tools never generate a menu path. Each step has a do part and a check part. In one by one mode a turn is do plus check. In all at once mode every step in the same chunk is joined into one turn and only the chunk's last check is used. Screen names are never translated or paraphrased.

Segment	Used for	LOB	Band
food_app_status	Compensation status in the app (Orders tile, Cancelled tab)	GrabFood	1

Segment	Used for	LOB	Band
food_app_history	Fallback when the order is not in the Cancelled tab	GrabFood	1
portal_status	Compensation status on the portal (GrabMart always)	Both	1
app_submit	Request for compensation link, form, fill	Both	1
portal_submit	Help Centre, route ladder, form, fill	Both	1 (route 1), 2 (routes 2, 3)
app_view_order	View cancelled order in the app, and the start of app submission when no order is open	Both	2
portal_view_order	View cancelled order on the portal	Both	2

```
// Code: Config (steps). Runs after Code: Config. Step text is Tier 1: said
exactly.
// {evidence} is replaced with cfg.evidence[lob]. No return statements.
context.uc4Cfg.steps = {
  app_view_order: [
    { id: "open_app", chunk: 1, do: "Please open the GrabMerchant app.",
check: "Can you see the home screen?" },
    { id: "orders_nav", chunk: 1, do: "Tap Orders on the bottom navigation
bar.", check: "Can you see the Orders screen?" },
    { id: "history_tab", chunk: 1, do: "Go to the History tab at the top of
the screen.", check: "Can you see it?" },
    { id: "date_selector", chunk: 1, do: "Tap the Date selector and choose
the date the order was cancelled.", check: "Have you selected the date?" },
    { id: "confirm_date", chunk: 1, do: "Tap Confirm to apply the date.",
check: "Did that work?" },
    { id: "order_id", chunk: 1, do: "Find and tap the specific Order ID to
view its details.", check: "Can you see the order details now?" }
  ],
  portal_view_order: [
    { id: "open_portal", chunk: 1, do: "Please open the GrabMerchant portal
on your computer.", check: "Can you see the home screen?" },
    { id: "orders_menu", chunk: 1, do: "Click Orders on the left navigation
menu.", check: "Can you see it?" },
    { id: "store_name", chunk: 1, do: "Select the relevant Store Name.",
```

```
check: "Have you selected your store?" },
    { id: "history_tab", chunk: 1, do: "Click the History tab.", check: "Can
you see it?" },
    { id: "date_filter", chunk: 1, do: "Click the Date Filter and select the
date the order was cancelled.", check: "Have you selected the date?" },
    { id: "status_cancelled", chunk: 1, do: "Click the status dropdown next
to the date, which shows All by default, and select Cancelled.", check: "Did
that work?" },
    { id: "order_id", chunk: 1, do: "Find and click the specific Order ID to
view its details.", check: "Can you see the order details now?" }
  ],
  food_app_status: [
    { id: "open_app", chunk: 1, do: "Please open the GrabMerchant app.",
check: "Tell me when you see the home screen." },
    { id: "orders_tile", chunk: 1, do: "Open the Orders tile.", check: "Tell
me when you see it." },
    { id: "cancelled_tab", chunk: 1, do: "Tap the Cancelled tab at the top.
It shows this month's cancelled orders and their compensation status.",
check: "Can you see your order there, and what status does it show?" },
    { id: "order_id", chunk: 2, do: "Tap the Order ID to open its details.",
check: "Tell me when the details page opens." }
  ],
  food_app_history: [
    { id: "history_tab", chunk: 1, do: "Go to the History tab at the top.",
check: "Can you see it?" },
    { id: "date_selector", chunk: 1, do: "Tap the Date selector and choose
the date the order was cancelled.", check: "Have you selected the date?" },
    { id: "confirm_date", chunk: 1, do: "Tap Confirm to apply the date.",
check: "Can you see your order now?" }
  ],
  portal_status: [
    { id: "open_portal", chunk: 1, do: "Please open the GrabMerchant portal
on your computer.", check: "Tell me when you see the home screen." },
    { id: "finance", chunk: 1, do: "Open Finance on the left sidebar.",
check: "Tell me when it opens." },
    { id: "transactions", chunk: 1, do: "Click the Transactions tab.", check:
"Tell me when you see it." },
    { id: "period_store", chunk: 1, do: "Choose a time period that covers the
day of the order, and the applicable store.", check: "Tell me when that's
set." },
    { id: "category", chunk: 1, do: "Under the Category filter, choose
Compensation for cancelled orders.", check: "Tell me when that's selected."
},
```

```
    { id: "status_check", chunk: 1, do: "", check: "Does the cancelled order
appear there as compensated?" }
  ],
  app_submit: [
    { id: "rfc_link", chunk: 1, do: "On the order details page, tap the
Request for compensation link.", check: "Can you see it?" },
    { id: "form", chunk: 1, do: "It opens the Help Centre article Report a
payment dispute. You need to be logged in as an Owner or Manager to submit a
request.", check: "Can you see the form?" },
    { id: "fill", chunk: 2, do: "Please fill in the form with the order
details and attach {evidence}, then submit it.", check: "Tell me when you've
submitted it." }
  ],
  portal_submit: [
    { id: "open_portal", chunk: 1, optional: "portalOpened", do: "Please open
the GrabMerchant portal on your computer.", check: "Tell me when you see the
home screen." },
    { id: "help_centre", chunk: 1, do: "Click Help Centre on the left
navigation menu.", check: "Tell me when you're on the main Help Centre page."
},
    { id: "route", chunk: 1, variants: [
        { lead: "", do: "Type dispute into the Search Bar and select the
article Report a payment dispute." },
        { lead: "Since we couldn't find it that way, let's try another way.
", do: "Under Common Problems, select Payouts and Reports, then Payout
issues, then click Report a payment dispute." },
        { lead: "Let's try one more way. ", do: "Under Browse by Services,
select Payment Services, then Payment issues, then click Report a payment
dispute." }
      ], check: "Were you able to find it?" },
    { id: "form", chunk: 1, do: "The form sits below the article text. You
need to be logged in as an Owner or Manager to submit a request.", check:
"Can you see the form?" },
    { id: "fill", chunk: 2, do: "Please fill in the form with the order
details and attach {evidence}, then submit it.", check: "Tell me when you've
submitted it." }
  ]
};

// Walkthrough Tier 1: fixed lines that carry screen names or policy
context.uc4Cfg.walkFixed = {
  scroll: { do: "That form sits below the article text. Please scroll down to
the bottom of the page.", check: "Can you see the form now?" },
```

```
  roleReminder: "If the form still isn't showing, you're likely not logged in
with the right role. Only an Owner or Manager can submit a request. Would you
like to try logging in again with an Owner or Manager account?",
  roleRetryApp: "Please sign in again with an Owner or Manager account, then
open the same cancelled order and tap Request for compensation. Tell me if
the form is visible now.",
  roleRetryPortal: "Please sign in again with an Owner or Manager account,
then open the same Report a payment dispute page. Tell me when you can see
the form.",
  orderMissingPortal: "Change the time period so it covers the date the order
was cancelled, and keep the same store and Category. Can you see it now?",
  crossBuild: "Would you like me to guide you to submit it through the
GrabMerchant portal instead, for our support team to review further?",
  martPortalOnly: "This particular check has to be done on the GrabMerchant
portal on a computer.",
  statusBlankLead: "If no compensation status is showing, the next step is to
submit a Request for Compensation."
};

// Walkthrough Tier 2: guided, the LLM phrases these in the conversation
language
context.uc4Cfg.walkGuided = {
  statusBlank: "Explain that when no compensation status shows, the next step
is to submit a Request for Compensation for our support team to review
further. Keep that phrase. Ask if they would like help with that.",
  statusCompensated: "Tell them the order shows as compensated, so the
compensation has been processed.",
  softClose: "Say no problem, and that they are welcome to contact our
support team for further review.",
  portalSoftClose: "Apologise that none of the routes reached the form, and
say they are welcome to contact our support team for further review.",
  hold: "Acknowledge in a few words that they can take their time and you
will stay on the line. Ask nothing new.",
  clarify: "Say sorry they are still having trouble and ask what they see on
their screen. If they already described it, ask one diagnostic question about
it instead."
};
```

6.7 resolve_order_age

Decides whether the order is within or beyond the LOB threshold and returns the next move. It is the only place the date question is asked, so the two asks cap is enforced in code. Band 1: TC-001 to TC-005, TC-025 to TC-029. Band 2: TC-019, TC-021, TC-022, TC-042, TC-044, TC-045.

Tool description (for the LLM): Call this whenever the merchant tells you, or corrects, when the cancelled order was cancelled, or mentions a cancelled order without a date, or raises a second cancelled order. Convert their words to days_ago where you can: today, just now, this morning = 0; yesterday = 1; two days ago = 2; last week = 7. In Malay or Manglish: hari ini, tadi, pagi tadi = 0; semalam = 1; kelmarin = 2; dua hari lepas = 2; minggu lepas = 7. Use weekday only for a bare weekday name in the past week. Use calendar_date for a date such as 11 September. Also pass what they want to know if they said it.

Parameters (JSON schema)

```
{
  "type": "object",
  "properties": {
    "days_ago": { "type": "integer", "minimum": 0, "description": "Whole days
since the order was cancelled. Omit if unknown." },
    "weekday": { "type": "string", "enum": ["Mon", "Tue", "Wed", "Thu",
"Fri", "Sat", "Sun"], "description": "Only for a bare weekday name meaning
the most recent one before today. Isnin=Mon, Selasa=Tue, Rabu=Wed,
Khamis=Thu, Jumaat=Fri, Sabtu=Sat, Ahad=Sun." },
    "calendar_date": { "type": "string", "description": "A calendar date the
merchant gave, as YYYY-MM-DD. Assume the current year." },
    "unclear": { "type": "boolean", "description": "True if the answer cannot
be turned into a date, for example a while ago, not sure." },
    "is_correction": { "type": "boolean", "description": "True if the
merchant is correcting a date already given for the same order." },
    "is_new_order": { "type": "boolean", "description": "True if the merchant
raises a different, separate cancelled order." },
    "merchant_intent": { "type": "string", "enum": ["none", "timeline",
"status", "submit", "status_and_submit", "view_order"], "description": "What
the merchant has said they want, if anything." }
  },
  "additionalProperties": false
}
```

Decision logic

Situation	State change	next_action and content	Band
No usable date, fewer than 2 asks	dateAskCount + 1	ask_date: phrases.askDate1, then phrases.askDate2	2
No usable date after 2 asks	threshold = beyond, dateSource = assumed	ask_open_question, nothing said about the date	2
Intent submit, status_and_submit or view_order	walk.pendingJourney set	collect_preferences, say phrases.canSubmit	1
Within, timeline not yet given	educateDelivered = true	deliver_say verbatim: educate + photoReminder + udr	1
Within, timeline already given	none	steer_back, guided.alreadyShared	1
Beyond, intent timeline or status	educateDelivered = true, pendingJourney = check_status	deliver_say verbatim: educate + phrases.offerStatusCheck	1
Beyond, intent none	none	ask_open_question	1
Correction that moves the verdict	threshold reset	ask_open_question, silently	2
is_new_order	New order B, walk reset, prefs kept	phrases.separateOrder first, then the rows above	2

Code: resolve_order_age

```
// Tool: resolve_order_age. No return statements; the result leaves through
actions.output.
const a = input.aiAgent?.toolArgs || {};
if (!context.shortTermMemory) {
  context.shortTermMemory = {};
}
const u = context.shortTermMemory.uc4;
const cfg = context.uc4Cfg;
const fx = cfg.fixed;
const ph = fx.phrases;
const wd = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
```

```
const res = (n, say, extra) => Object.assign({ success: true, next_action: n,
say: say || "", guide: "", verbatim: false }, extra || {});
let out = res("apologise_and_retry_once", "");
let prefix = "";

// 1. Find or create the order
if (a.is_new_order === true || u.orders.length === 0) {
  const ref = String.fromCharCode(65 + u.orders.length); // A, B, C
  u.orders.push({ ref: ref, ageDays: -1, dateSource: "", threshold: "",
dateAskCount: 0,
    educateDelivered: false, followupAnswered: false, submitted: false,
closedAfterRole: false });
  u.activeOrder = ref;
  Object.assign(u.walk, { journey: "", pendingJourney: "", segment: "",
stepIndex: 0, stuckCount: 0, routeIndex: 0,
    scrollAsked: false, roleAttempts: 0, roleState: "", awaiting: "", then:
"", crossBuildOffered: false,
    orderDetailsOpen: false, status: "" });
  if (u.orders.length > 1) {
    prefix = ph.separateOrder + " ";
  }
}
const o = u.orders.find(x => x.ref === u.activeOrder);

// 2. Work out the age, or read the weekday verdict directly
let verdict = "";
if (Number.isInteger(a.days_ago)) {
  o.ageDays = a.days_ago;
  o.dateSource = "days";
} else if (a.calendar_date) {
  const diff = Math.round((Date.parse(u.today.iso) -
Date.parse(a.calendar_date)) / 86400000);
  if (diff >= 0) {
    o.ageDays = diff;
    o.dateSource = "date";
  }
} else if (a.weekday) {
  const within = cfg.weekdayWithin[u.lob];
  verdict = (within === "any" || within[u.today.weekday].indexOf(a.weekday)
!== -1) ? "within" : "beyond";
  const back = (wd.indexOf(u.today.weekday) - wd.indexOf(a.weekday) + 7) % 7;
  o.ageDays = back === 0 ? 7 : back;
  o.dateSource = "weekday";
```

```
}
if (!verdict && o.ageDays >= 0) {
  verdict = o.ageDays <= cfg.thresholdDays[u.lob][u.today.weekday] ? "within"
: "beyond";
}
if (verdict) {
  o.threshold = verdict;
}

const intent = a.merchant_intent || "none";
const wantsGuide = (intent === "submit" || intent === "status_and_submit" ||
intent === "view_order");

// 3. Decide the next move
if (o.threshold === "" && o.dateAskCount < cfg.flags.maxDateAsks) {
  o.dateAskCount += 1;
  out = res("ask_date", prefix + (o.dateAskCount === 1 ? ph.askDate1 :
ph.askDate2));
} else if (o.threshold === "") {
  o.threshold = "beyond";
  o.dateSource = "assumed";
  out = res("ask_open_question", prefix + ph.openQuestion,
    { guide: "The date is settled. Never ask it again and never mention any
assumption." });
} else if (wantsGuide) {
  u.walk.pendingJourney = intent === "submit" ? "submit_request" : (intent
=== "view_order" ? "view_order" : "status_then_submit");
  out = res("collect_preferences", prefix + (intent === "view_order" ? "" :
ph.canSubmit));
} else if (o.threshold === "within" && o.educateDelivered) {
  out = res("steer_back", "", { guide: cfg.guided.alreadyShared });
} else if (o.threshold === "within") {
  o.educateDelivered = true;
  out = res("deliver_say", prefix + fx[u.lob].educate + " " +
fx[u.lob].photoReminder + " " + fx.udr, { verbatim: true });
} else if (intent === "timeline" || intent === "status") {
  o.educateDelivered = true;
  u.walk.pendingJourney = "check_status";
  out = res("deliver_say", prefix + fx[u.lob].educate + " " +
ph.offerStatusCheck, { verbatim: true });
} else {
  out = res("ask_open_question", prefix + ph.openQuestion,
    { guide: a.is_correction ? "Date corrected. Do not mention the change or
```

```
any day count." : "" });
}

input.uc4Result = out;
actions.output("", out);
```

Unit checks (set cfg.testToday in Code: Config)

LOB	Today	Input	Expected
GrabFood	Wed	days_ago 2	within
GrabFood	Wed	days_ago 3	beyond
GrabFood	Mon	days_ago 4	within
GrabFood	Tue	weekday Fri	within
GrabFood	Tue	weekday Tue	beyond
GrabMart	Tue	days_ago 5	within
GrabMart	Sun	days_ago 10	beyond
Either	any	unclear twice	ask_date twice, then ask_open_question with threshold beyond

6.8 get_approved_answer

Returns Grab approved wording for known follow up topics, handles language requests, and is the single entry point for every offer of a person that is not caused by a walkthrough failure. Routing offers through one tool means uc4.transfer.offered is always set before transfer_to_live_agent becomes available. Band 1: TC-002, TC-010, TC-026. Band 2: TC-020, TC-023, TC-024, TC-043 and the Malay conversation run.

Tool description (for the LLM): Call this when the merchant asks a follow up after the compensation timeline, asks about evidence, says they have no photo, says the order was cancelled before it was prepared, raises something outside cancelled order compensation, says payout without linking it to a cancelled order, asks for the other language or keeps speaking it, asks for a language you do not speak, is unhappy with you, asks for a person, or says the order is actually the other brand.

Parameters (JSON schema)

```
{
  "type": "object",
  "properties": {
    "topic": {
      "type": "string",
      "enum": ["further_inquiry", "evidence_question", "photo_missing",
"not_prepared", "out_of_scope",
               "payout_unclear", "language_request", "dissatisfied",
"human_request", "other_brand"],
      "description": "The topic that best matches what the merchant just
said."
    },
    "language": {
      "type": "string",
      "enum": ["en", "ms", "other"],
      "description": "For language_request only. en = English, ms = Bahasa
Melayu, other = any other language. Manglish is not a language request."
    }
  },
  "required": ["topic"],
  "additionalProperties": false
}
```

Topic table

Topic	Tier	next_action	Content	Band
further_inquiry, first time for this order	Fixed	deliver_say	fixed.furtherInquiry (GrabMart beyond: partial compensation line first). Sets pendingJourney = view_order.	1
further_inquiry, already answered	Guided	deliver_guided	guided.alreadyShared	1
out_of_scope	Fixed	offer_transfer	fixed.outOfScopeLead + fixed.offerPerson (TC-010)	1
evidence_question, photo_missing, not_prepared, payout_unclear	Guided	deliver_guided	The matching guide	2

Topic	Tier	next_action	Content	Band
language_request, same language as now	Guided	steer_back	Carry on	2
language_request, en or ms, switch mode	Fixed	deliver_say	phrases.switched in the new language; pack swapped; input.uc4LangChanged = true	2
language_request, other language or ask_then_transfer mode	Fixed	deliver_say, then offer_transfer	First ask: phrases.stayInLanguage (ask_then_transfer) or phrases.languagesOffered (other). Second: phrases.understand + offerPerson (TC-023, TC-024)	2
dissatisfied first, human_request with no attempt yet	Guided	deliver_guided	guided.dissatisfiedFirst or guided.humanFirst	2
dissatisfied again, human_request after an attempt	Fixed	offer_transfer	offerPerson (human_request reopens after a decline)	2
other_brand	Fixed	offer_transfer	fixed.otherBrand	2

An offer sets transfer.offered and transfer.reason. After a decline the offer is suppressed and guided.declinedPerson is returned, unless the merchant asks for a person in their own words.

Flow under the tool. Code: get_approved_answer, then an If on input.uc4LangChanged. When true (Band 2), a second If on uc4.lang runs the Malay or the English Set Session Config node so speech and voice follow the new language. Both branches end in Resolve Tool Action.

Code: get_approved_answer

```
// Tool: get_approved_answer. Fixed lines, guided intents and language
requests. No return statements.
const a = input.aiAgent?.toolArgs || {};
if (!context.shortTermMemory) {
```

```
  context.shortTermMemory = {};
}
const u = context.shortTermMemory.uc4;
const cfg = context.uc4Cfg;
const fx = cfg.fixed;
const gd = cfg.guided;
const o = u.orders.find(x => x.ref === u.activeOrder) || {};
const res = (n, say, extra) => Object.assign({ success: true, next_action: n,
say: say || "", guide: "", verbatim: false }, extra || {});
const guided = text => res("deliver_guided", "", { guide: text.replace("
{evidence}", cfg.evidence[u.lob]).replace("{lob}", u.lob) });
let out = res("apologise_and_retry_once", "");
input.uc4LangChanged = false;

// An offer is suppressed after a decline, unless the merchant asks for a
person themselves
const makeOffer = (reason, lead, reopen) => {
  if (u.transfer.declined && !reopen) {
    out = guided(gd.declinedPerson);
  } else {
    u.transfer.offered = true;
    u.transfer.reason = reason;
    u.transfer.declined = false;
    out = res("offer_transfer", (lead + " " + fx.offerPerson).trim(), {
verbatim: true });
  }
};
const supported = a.language === "en" || a.language === "ms";

const t = a.topic;
if (t === "further_inquiry" && o.followupAnswered) {
  out = guided(gd.alreadyShared);
} else if (t === "further_inquiry") {
  o.followupAnswered = true;
  u.walk.pendingJourney = "view_order";
  const lead = (u.lob === "GrabMart" && o.threshold === "beyond") ?
fx.GrabMart.partialDisclaimer + " " : "";
  out = res("deliver_say", lead + fx.furtherInquiry, { verbatim: true });
} else if (t === "evidence_question") {
  out = guided(gd.evidence);
} else if (t === "photo_missing") {
  out = guided(gd.photoMissing);
} else if (t === "not_prepared") {
```

```
  out = guided(gd.notPrepared);
} else if (t === "payout_unclear") {
  out = guided(gd.payoutUnclear);
} else if (t === "out_of_scope") {
  makeOffer("out_of_scope", fx.outOfScopeLead, false);
} else if (t === "language_request" && supported && a.language === u.lang) {
  out = res("steer_back", "", { guide: "You are already in this language.
Carry on where the call left off." });
} else if (t === "language_request" && supported && cfg.flags.languageSwitch
=== "switch") {
  // Switch the conversation language: new pack for the tools, new voice
settings in the Flow
  u.lang = a.language;
  u.langName = a.language === "ms" ? "Bahasa Melayu" : "English";
  u.language.switches += 1;
  const pack = cfg.content[u.lang];
  Object.assign(cfg, { fixed: pack.fixed, steps: pack.steps, walkFixed:
pack.walkFixed,
    messages: pack.messages, evidence: pack.evidence });
  input.uc4LangChanged = true;
  out = res("deliver_say", pack.fixed.phrases.switched, { verbatim: true,
    guide: "From now on reply only in " + u.langName + ". After this line,
re-ask the question that was open, if any." });
} else if (t === "language_request") {
  // Unsupported language, or ask_then_transfer mode: one ask, then an offer
of a person
  u.language.otherAsks += 1;
  if (u.language.otherAsks === 1) {
    out = res("deliver_say", supported ? fx.phrases.stayInLanguage :
fx.phrases.languagesOffered, { verbatim: true });
  } else {
    makeOffer("language", fx.phrases.understand, false);
  }
} else if (t === "dissatisfied" && u.dissatisfaction.attempts === 0) {
  u.dissatisfaction.attempts = 1;
  out = guided(gd.dissatisfiedFirst);
} else if (t === "dissatisfied") {
  makeOffer("dissatisfied", fx.phrases.sorryNotHelped, false);
} else if (t === "human_request" && u.dissatisfaction.attempts === 0) {
  u.dissatisfaction.attempts = 1;
  out = guided(gd.humanFirst);
} else if (t === "human_request") {
  makeOffer("human_request", "", true);
```

```
} else if (t === "other_brand") {
  u.transfer.offered = true;
  u.transfer.reason = "other_brand";
  out = res("offer_transfer", fx.otherBrand.replace("{lob}", u.lob), {
verbatim: true });
}

input.uc4Result = out;
actions.output("", out);
```

6.9 set_guidance_preference

Collects the three guidance preferences one per turn, in the order platform, style, loudspeaker, and hands over to start_walkthrough once platform and style are known. Each question is asked at most twice (loudspeaker once). If the merchant pushes for the steps or ignores a question twice, the tool assumes app, all at once and no loudspeaker, and the agent says so in one short clause. Band 1: TC-003, TC-005 to TC-009, TC-027, TC-029.

Tool description (for the LLM): Call this after the merchant agrees to be shown or guided, or asks to submit a request, and again with each answer they give to a preference question. Pass only what the merchant actually said. A device mention (I'm on my phone) is not a request to be guided. Malay and Manglish answers: app, apps, telefon, handphone = app; portal, komputer, laptop = portal; satu satu, step by step = one_by_one; sekali gus, semua sekali = all_at_once; tak payah, tak perlu = no (for loudspeaker).

Parameters (JSON schema)

```
{
  "type": "object",
  "properties": {
    "journey": { "type": "string", "enum": ["view_order", "check_status",
"submit_request", "status_then_submit"], "description": "What the merchant
agreed to be guided through. Omit to keep the pending journey." },
    "platform": { "type": "string", "enum": ["app", "portal", "either"],
"description": "GrabMerchant app or GrabMerchant portal. Heard as add, apps,
potter, porter, portol are valid." },
    "style": { "type": "string", "enum": ["one_by_one", "all_at_once",
"either"] },
    "loudspeaker": { "type": "string", "enum": ["yes", "no"] },
    "pressed_for_steps": { "type": "boolean", "description": "True if the
merchant refused the setup questions, for example just tell me, don't ask so
much." }
  },
  "additionalProperties": false
}
```

Logic

1. Store the journey if given, else use walk.pendingJourney. If neither exists, return steer_back.
2. Store any preference given. "either" sets the default (app, all_at_once).
3. pressed_for_steps: fill every missing value with its default and return start_walkthrough, with a guide asking the agent to mention the assumption in one clause.
4. Otherwise find the first missing value in the order platform, style, loudspeaker. If its ask count is below the limit (2, loudspeaker 1), ask it (ask_preference, verbatim). If the limit is reached, assume the default and move on.
5. When nothing is left to ask, return start_walkthrough with the journey in guide.

Preference	Question (fixed.phrases)	Default when assumed
platform	askPlatform: "Are you on the GrabMerchant app, or the GrabMerchant portal on a computer?"	app
style	askStyle: "Would you like the steps one by one, or all at once?"	all_at_once

Preference	Question (fixed.phrases)	Default when assumed
loudspeaker	askLoudspeaker: "Would you like to put me on loudspeaker before we start?"	no

Preferences hold for the whole call, including a second order. No navigation step is given before the platform question has been asked at least once.

Code: set_guidance_preference

```
// Tool: set_guidance_preference. No return statements; the result leaves
through actions.output.
const a = input.aiAgent?.toolArgs || {};
if (!context.shortTermMemory) {
  context.shortTermMemory = {};
}
const u = context.shortTermMemory.uc4;
const p = u.prefs;
const ph = context.uc4Cfg.fixed.phrases;
const max = context.uc4Cfg.flags.maxPrefAsks;
const res = (n, say, extra) => Object.assign({ success: true, next_action: n,
say: say || "", guide: "", verbatim: false }, extra || {});
const Q = { platform: ph.askPlatform, style: ph.askStyle, loudspeaker:
ph.askLoudspeaker };
const DEF = { platform: "app", style: "all_at_once", loudspeaker: "no" };
let out = res("apologise_and_retry_once", "");

if (a.journey) { u.walk.pendingJourney = a.journey; }
if (a.platform) { p.platform = a.platform === "either" ? DEF.platform :
a.platform; }
if (a.style) { p.style = a.style === "either" ? DEF.style : a.style; }
if (a.loudspeaker) { p.loudspeaker = a.loudspeaker; }

if (!u.walk.pendingJourney) {
  out = res("steer_back", "");
} else if (a.pressed_for_steps === true) {
  const platformAssumed = !p.platform;
  ["platform", "style", "loudspeaker"].forEach(k => {
    if (!p[k]) { p[k] = DEF[k]; p.assumed.push(k); }
  });
  out = res("start_walkthrough", "", { guide: "journey=" +
u.walk.pendingJourney +
```

```
    (platformAssumed ? ". In one short clause, say you will assume the app
and give all the steps at once, and that they can say if they are on the
portal." : "") });
} else {
  // First missing preference, in order. Loudspeaker is asked once only.
  let asked = false;
  const assumedNow = [];
  ["platform", "style", "loudspeaker"].forEach(k => {
    const limit = k === "loudspeaker" ? 1 : max;
    if (!asked && !p[k] && p.asks[k] < limit) {
      p.asks[k] += 1;
      asked = true;
      out = res("ask_preference", Q[k], { verbatim: true });
    } else if (!asked && !p[k] && k !== "loudspeaker") {
      p[k] = DEF[k];
      p.assumed.push(k);
      assumedNow.push(k === "platform" ? "the app" : "all the steps at
once");
    }
  });
  if (!asked) {
    out = res("start_walkthrough", "", { guide: "journey=" +
u.walk.pendingJourney +
      (assumedNow.length > 0 ? ". In one short clause, say you will assume "
+ assumedNow.join(" and ") + ", and that they can correct you." : "") });
  }
}

input.uc4Result = out;
actions.output("", out);
```

6.10 start_walkthrough

Picks the step segment from the LOB, journey and platform, applies the GrabMart rules, and delivers the first step (or the first chunk in all at once mode). It is the only place a segment is chosen, so GrabMart can never be sent to the app Cancelled tab for a status check. Band 1: TC-003, TC-005 to TC-009, TC-027, TC-029. Band 2: TC-017, TC-040.

Tool description (for the LLM): Call this when set_guidance_preference returns start_walkthrough, or when the merchant accepts help submitting a request after viewing their order or checking status. Pass the journey from the previous tool's guide.

Parameters (JSON schema)

```
{
  "type": "object",
  "properties": {
    "journey": { "type": "string", "enum": ["view_order", "check_status",
"submit_request", "status_then_submit"] }
  },
  "required": ["journey"],
  "additionalProperties": false
}
```

Segment selection

Journey	Platform	GrabFood	GrabMart	Band
check_status, status_then_submit	app	food_app_status	portal_status, after walkFixed.martPortalOnly	1
check_status, status_then_submit	portal	portal_status	portal_status	1
submit_request	app	app_submit if order details are open, else app_view_order then app_submit	same	1
submit_request	portal	portal_submit	portal_submit	1
view_order	app or portal	app_view_order or portal_view_order	same	2

GrabMart: the partial compensation line is said once per call, verbatim, before the first status or submission step. Already submitted guard: if the active order was submitted or closed with Final Educate, a submission journey returns guided.alreadySubmitted.

Code: start_walkthrough

```
// Tool: start_walkthrough. No return statements; the result leaves through
actions.output.
const a = input.aiAgent?.toolArgs || {};
if (!context.shortTermMemory) {
  context.shortTermMemory = {};
}
```

```
const u = context.shortTermMemory.uc4;
const cfg = context.uc4Cfg;
const p = u.prefs;
const w = u.walk;
const wf = cfg.walkFixed;
const o = u.orders.find(x => x.ref === u.activeOrder) || {};
const ev = cfg.evidence[u.lob];
const res = (n, say, extra) => Object.assign({ success: true, next_action: n,
say: say || "", guide: "", verbatim: false }, extra || {});
const journey = a.journey || w.pendingJourney;
const submits = journey === "submit_request" || journey ===
"status_then_submit";
let out = res("apologise_and_retry_once", "");

if (submits && (o.submitted || o.closedAfterRole)) {
  out = res("deliver_guided", "", { guide: cfg.guided.alreadySubmitted });
} else {
  // 1. Choose the segment
  let lead = "";
  w.journey = journey;
  w.platform = p.platform;
  w.then = "";
  if (journey === "view_order") {
    w.segment = p.platform === "portal" ? "portal_view_order" :
"app_view_order";
  } else if (journey === "check_status" || journey === "status_then_submit")
{
    if (u.lob === "GrabFood" && p.platform === "app") {
      w.segment = "food_app_status";
    } else {
      w.segment = "portal_status";
      w.platform = "portal";
      if (u.lob === "GrabMart" && p.platform === "app") { lead =
wf.martPortalOnly + " "; }
    }
    if (journey === "status_then_submit") { w.then = "submit"; }
  } else if (p.platform === "portal") {
    w.segment = "portal_submit";
  } else if (w.orderDetailsOpen) {
    w.segment = "app_submit";
  } else {
    w.segment = "app_view_order";
    w.then = "app_submit";
```

```
  }
  if (u.lob === "GrabMart" && journey !== "view_order" && !w.disclaimerSaid)
{
    lead = cfg.fixed.GrabMart.partialDisclaimer + " " + lead;
    w.disclaimerSaid = true;
  }
  Object.assign(w, { stepIndex: 0, stuckCount: 0, routeIndex: 0, scrollAsked:
false,
    roleAttempts: 0, roleState: "", awaiting: "", status: "active" });
  u.outcome.journeys.push(journey);

  // 2. Deliver the first step, or the first chunk in all at once mode
  const steps = cfg.steps[w.segment];
  const txt = s => (s.variants ? s.variants[w.routeIndex].lead +
s.variants[w.routeIndex].do : s.do).replace("{evidence}", ev);
  const skip = s => !!(s && s.optional && w[s.optional]);
  while (skip(steps[w.stepIndex])) { w.stepIndex += 1; }
  let say = "";
  if (p.style === "one_by_one") {
    say = txt(steps[w.stepIndex]) + " " + steps[w.stepIndex].check;
  } else {
    const chunk = steps[w.stepIndex].chunk;
    const parts = [];
    let i = w.stepIndex;
    while (i < steps.length && steps[i].chunk === chunk) {
      if (!skip(steps[i])) { parts.push(txt(steps[i])); }
      i += 1;
    }
    w.stepIndex = i - 1;
    say = parts.join(" ") + " " + steps[w.stepIndex].check;
  }
  if (w.segment.indexOf("portal") !== -1) { w.portalOpened = true; }
  out = res("deliver_step", (lead + say).trim(), { verbatim: true, step_id:
steps[w.stepIndex].id,
    guide: "Read screen names exactly. Wait for the merchant, then call
advance_walkthrough." });
}

input.uc4Result = out;
actions.output("", out);
```

6.11 advance_walkthrough

The walkthrough state machine. The AI Agent classifies each merchant reply into one outcome and the tool decides everything else: the next step, repeats, clarifying questions, the next portal route, the scroll and role path, the app to portal fallback, submission and offers of a person. Band 1: TC-003, TC-005 to TC-009, TC-027, TC-029. Band 2: TC-013 to TC-018, TC-036 to TC-041.

Tool description (for the LLM): Call this on every merchant reply while a walkthrough is active. Choose the one outcome that matches what they said about the step you just gave. A yes or no to a question you asked (help submitting, use the portal instead, log in again) is yes_to_question or no_to_question. If they say wait, they are checking, or their device is slow, use needs_time. Malay and Manglish: dah, sudah, ok, boleh, nampak = confirmed; tak nampak, takde, tak jumpa = not visible or not found; jap, kejap, tunggu = needs_time; tak payah, tak nak = no_to_question or declined_walkthrough.

Parameters (JSON schema)

```
{
  "type": "object",
  "properties": {
    "outcome": {
      "type": "string",
      "enum": ["confirmed", "failed", "order_not_visible", "status_blank",
"status_compensated",
               "link_not_found", "route_not_found", "form_not_visible",
"scrolled_no_form", "form_visible",
               "submitted", "yes_to_question", "no_to_question",
"platform_switch", "skip_to_step",
               "needs_time", "declined_walkthrough"]
    },
    "step_id": { "type": "string", "description": "For skip_to_step or
failed: the step the merchant means, if they named one." },
    "new_platform": { "type": "string", "enum": ["app", "portal"],
"description": "For platform_switch only." },
    "screen_description": { "type": "string", "description": "What the
merchant says is on their screen, if they described it." }
  },
  "required": ["outcome"],
  "additionalProperties": false
}
```

Outcome table

Outcome	Where	Result	Band
confirmed	Any step	stuckCount = 0. Next step, or the segment completion rule below.	1
failed	Any step except a portal route	stuckCount + 1, then the stuckLadder rung: repeat (same step, add where it is), clarify, offer_transfer (reason difficulty_cap)	1: first rung. 2: full ladder
needs_time	Any	hold. No counter changes.	1
order_not_visible	food_app_status	Switch to food_app_history (History tab, Date selector, Confirm), then back to order_id	1
order_not_visible	portal_status	walkFixed.orderMissingPortal (change the time period)	1
status_blank	status segments	status_then_submit: straight to submission. Otherwise walkGuided.statusBlank and wait for yes or no.	1
status_compensated	status segments	walkGuided.statusCompensated, then the anything else question	1
link_not_found	app_submit rfc_link	First time: walkFixed.crossBuild. Yes: portal_submit. No: soft close.	1
route_not_found	portal_submit route	Next route with its lead. After route 3: soft close (portalExhaustedAction = soft_close)	1: route 1. 2: routes 2, 3
form_not_visible	form step	First time: scroll. After that: as scrolled_no_form.	1
scrolled_no_form	form step	roleReminder with a retry question. Yes: retry instruction. Retry fails: offer_transfer (role_failed). No: Final Educate.	1
form_visible	form step	Go to the fill step	1

Outcome	Where	Result	Band
submitted	fill step	order.submitted = true, afterSubmission, anything else question	1
yes_to_question, no_to_question	after a question	Read by walk.awaiting: submit_choice, cross_build or role_retry	1
platform_switch, skip_to_step	Any	Restart the equivalent segment on the new platform, or jump to the named step	2
declined_walkthrough	Any	Walk completed, anything else question	1

Segment completion: app_view_order, portal_view_order and food_app_status end with order details open; if then = app_submit, app_submit starts, else fixed.orderVisible offers submission. food_app_history returns to food_app_status at order_id. portal_status answered yes means compensated. app_submit and portal_submit end as submitted.

Code: advance_walkthrough

```
// Tool: advance_walkthrough. No return statements: every helper writes to
out.
const a = input.aiAgent?.toolArgs || {};
if (!context.shortTermMemory) {
  context.shortTermMemory = {};
}
const u = context.shortTermMemory.uc4;
const cfg = context.uc4Cfg;
const p = u.prefs;
const w = u.walk;
const fx = cfg.fixed;
const wf = cfg.walkFixed;
const wg = cfg.walkGuided;
const o = u.orders.find(x => x.ref === u.activeOrder) || {};
const ev = cfg.evidence[u.lob];
const steps = () => cfg.steps[w.segment] || [];
const cur = () => steps()[w.stepIndex];
const idx = id => steps().findIndex(s => s.id === id);
const res = (n, say, extra) => Object.assign({ success: true, next_action: n,
```

```
say: say || "", guide: "",
  verbatim: false, step_id: cur() ? cur().id : "" }, extra || {});
const txt = s => (s.variants ? s.variants[w.routeIndex].lead +
s.variants[w.routeIndex].do : s.do).replace("{evidence}", ev);
const skip = s => !!(s && s.optional && w[s.optional]);
let out = res("apologise_and_retry_once", "");

// Helpers. None of them use return; each one sets out.
const deliver = lead => {
  while (skip(steps()[w.stepIndex])) { w.stepIndex += 1; }
  const list = steps();
  let say = "";
  if (p.style === "one_by_one") {
    say = txt(list[w.stepIndex]) + " " + list[w.stepIndex].check;
  } else {
    const ch = list[w.stepIndex].chunk;
    const parts = [];
    let i = w.stepIndex;
    while (i < list.length && list[i].chunk === ch) {
      if (!skip(list[i])) { parts.push(txt(list[i])); }
      i += 1;
    }
    w.stepIndex = i - 1;
    say = parts.join(" ") + " " + list[w.stepIndex].check;
  }
  if (w.segment.indexOf("portal") !== -1) { w.portalOpened = true; }
  out = res("deliver_step", ((lead || "") + say).trim(), { verbatim: true });
};
const startSegment = (seg, lead) => {
  Object.assign(w, { segment: seg, stepIndex: 0, stuckCount: 0, routeIndex:
0, scrollAsked: false,
    roleAttempts: 0, roleState: "", awaiting: "" });
  if (seg.indexOf("portal") !== -1) { w.platform = "portal"; }
  deliver(lead);
};
const finish = (guideText, fixedText) => {
  w.status = "completed";
  w.awaiting = "";
  if (fixedText) {
    out = res("deliver_say", fixedText + " " + fx.udr, { verbatim: true });
  } else {
    out = res("deliver_guided", "", { guide: guideText + " Then ask if there
is anything else you can help with." });
```

```
  }
};
const markSubmitted = () => {
  o.submitted = true;
  u.outcome.result = "submitted";
  finish("", fx.afterSubmission);
};
const offer = (reason, lead) => {
  if (u.transfer.declined) {
    finish(wg.softClose, "");
  } else {
    u.transfer.offered = true;
    u.transfer.reason = reason;
    out = res("offer_transfer", ((lead || "") + fx.offerPerson).trim(), {
verbatim: true });
  }
};
const toSubmit = lead => {
  if (w.platform === "portal") {
    startSegment("portal_submit", lead);
  } else if (w.orderDetailsOpen) {
    startSegment("app_submit", lead);
  } else if (w.segment === "food_app_status") {
    w.then = "app_submit";
    w.stepIndex = idx("order_id");
    deliver(lead);
  } else {
    w.then = "app_submit";
    startSegment("app_view_order", lead);
  }
};
const segmentEnd = () => {
  const seg = w.segment;
  if (seg === "food_app_history") {
    w.segment = "food_app_status";
    w.stepIndex = idx("order_id");
    deliver();
  } else if (seg === "portal_status") {
    finish(wg.statusCompensated, "");
  } else if (seg === "app_submit" || seg === "portal_submit") {
    markSubmitted();
  } else if (w.then === "app_submit") {
    w.orderDetailsOpen = true;
```

```
    w.then = "";
    startSegment("app_submit");
  } else {
    w.orderDetailsOpen = true;
    w.awaiting = "submit_choice";
    out = res("deliver_say", fx.orderVisible, { verbatim: true });
  }
};
const roleStep = () => {
  if (w.roleState === "" && !cfg.flags.roleRetryEnabled) {
    offer("role_failed", fx.phrases.ownerManagerOnly + " ");
  } else if (w.roleState === "" || w.roleAttempts < cfg.flags.roleMaxAttempts
- 1) {
    w.roleState = "asked";
    w.awaiting = "role_retry";
    out = res("deliver_say", wf.roleReminder, { verbatim: true });
  } else {
    offer("role_failed", "");
  }
};

// Main decision chain
if (a.step_id && idx(a.step_id) !== -1) { w.stepIndex = idx(a.step_id); }
let oc = a.outcome;
if (oc === "order_not_visible" && w.segment !== "food_app_status" &&
w.segment !== "portal_status") {
  oc = "failed"; // on view segments a missing order is a failed date step
}

if (oc === "confirmed") {
  w.stuckCount = 0;
  if (w.stepIndex >= steps().length - 1) { segmentEnd(); } else { w.stepIndex
+= 1; deliver(); }
} else if (oc === "failed") {
  w.stuckCount += 1;
  const ladder = cfg.flags.stuckLadder;
  const act = ladder[Math.min(w.stuckCount - 1, ladder.length - 1)];
  if (act === "repeat") {
    out = res("deliver_step", txt(cur()) + " " + cur().check,
      { verbatim: true, guide: "Repeat this one step. If they asked where,
add where it is on the screen." });
  } else if (act === "clarify") {
    out = res("deliver_guided", "", { guide: wg.clarify +
```

```
(a.screen_description ? " They said: " + a.screen_description : "") });
  } else {
    offer("difficulty_cap", fx.phrases.sorryStuck + " ");
  }
} else if (oc === "needs_time") {
  out = res("hold", "", { guide: wg.hold });
} else if (oc === "order_not_visible" && w.segment === "food_app_status") {
  const keepThen = w.then;
  startSegment("food_app_history");
  w.then = keepThen;
} else if (oc === "order_not_visible") {
  out = res("deliver_step", wf.orderMissingPortal, { verbatim: true });
} else if (oc === "status_blank" && w.journey === "status_then_submit") {
  toSubmit(wf.statusBlankLead + " ");
} else if (oc === "status_blank") {
  w.awaiting = "submit_choice";
  out = res("deliver_guided", "", { guide: wg.statusBlank });
} else if (oc === "status_compensated") {
  finish(wg.statusCompensated, "");
} else if (oc === "link_not_found" && !w.crossBuildOffered) {
  w.crossBuildOffered = true;
  w.awaiting = "cross_build";
  out = res("deliver_say", wf.crossBuild, { verbatim: true });
} else if (oc === "link_not_found") {
  finish(wg.softClose, "");
} else if (oc === "route_not_found" && w.routeIndex < 2) {
  w.routeIndex += 1;
  w.stepIndex = idx("route");
  deliver();
} else if (oc === "route_not_found" && cfg.flags.portalExhaustedAction ===
"offer_transfer") {
  offer("portal_exhausted", fx.phrases.noneWorked + " ");
} else if (oc === "route_not_found") {
  finish(wg.portalSoftClose, "");
} else if (oc === "form_not_visible" && !w.scrollAsked) {
  w.scrollAsked = true;
  out = res("deliver_step", wf.scroll.do + " " + wf.scroll.check, { verbatim:
true });
} else if (oc === "form_not_visible" || oc === "scrolled_no_form") {
  w.scrollAsked = true;
  roleStep();
} else if (oc === "form_visible") {
  w.stepIndex = idx("fill");
```

```
  deliver();
} else if (oc === "submitted") {
  markSubmitted();
} else if (oc === "yes_to_question" && w.awaiting === "submit_choice") {
  w.awaiting = "";
  toSubmit();
} else if (oc === "yes_to_question" && w.awaiting === "cross_build") {
  w.awaiting = "";
  startSegment("portal_submit");
} else if (oc === "yes_to_question" && w.awaiting === "role_retry") {
  w.awaiting = "";
  w.roleAttempts += 1;
  out = res("deliver_step", w.platform === "portal" ? wf.roleRetryPortal :
wf.roleRetryApp, { verbatim: true });
} else if (oc === "no_to_question" && w.awaiting === "role_retry") {
  o.closedAfterRole = true;
  finish(cfg.guided.finalEducate, "");
} else if (oc === "no_to_question") {
  finish(w.awaiting === "cross_build" ? wg.softClose : "Acknowledge
briefly.", "");
} else if (oc === "platform_switch") {
  p.platform = a.new_platform || p.platform;
  w.platform = p.platform;
  const map = p.platform === "portal"
    ? { app_view_order: "portal_view_order", food_app_status:
"portal_status", food_app_history: "portal_status", app_submit:
"portal_submit" }
    : { portal_view_order: "app_view_order", portal_status: u.lob ===
"GrabFood" ? "food_app_status" : "portal_status", portal_submit: "app_submit"
};
  startSegment(map[w.segment] || w.segment);
} else if (oc === "skip_to_step") {
  deliver();
} else if (oc === "declined_walkthrough") {
  finish("Acknowledge briefly.", "");
} else if (oc === "yes_to_question") {
  out = res("steer_back", "");
}

input.uc4Result = out;
actions.output("", out);
```

Developer notes

- The whole decision chain is built in Band 1, because the branches share helpers. Band 2 branches are tested later.
- In all at once mode a failure report points at the last step of the chunk unless the agent passes step_id.
- If Grab confirms an offer of a person after all three portal routes fail, set portalExhaustedAction to offer_transfer. No code change.

6.12 transfer_to_live_agent (mock)

Acts on the merchant's answer to an offer of a person. The guard (transfer.offered === true) means it is available only after a tool has made the offer. On yes it is terminal: the fixed transfer message plays, the ACW summary is written to the log with classification 'others', and the session ends. No CXone transfer happens in this phase (E-05). Band 1: TC-009, TC-010. Band 2: TC-018, TC-020, TC-024, TC-041, TC-043.

Tool description (for the LLM): Call this when the merchant answers your offer to connect them with someone. yes only if the whole sense of their reply is yes, put me through. A reply that only reports the step still failing is unclear, not yes. Phrases such as no need, nevermind, I'll do it later, tak payah, tak nak are no.

Parameters (JSON schema)

```
{
  "type": "object",
  "properties": {
    "merchant_answer": { "type": "string", "enum": ["yes", "no", "unclear"] }
  },
  "required": ["merchant_answer"],
  "additionalProperties": false
}
```

Answer	Reason on file	Result
yes	any	Terminal: Build ACW, Say messages.transfer, Code: Mock Transfer, Hang Up
no	language	next_action end_call, exit_type language_not_supported
no	other_brand	steer_back, guided.continueOrder
no	any other	deliver_guided, guided.declinedPerson

Answer	Reason on file	Result
unclear, first time	any	Ask once more in different words
unclear, second time	any	Treated as no

On every no: transfer.declined = true and transfer.offered = false, which removes the tool until a new offer is made.

Flow under the tool

1. Code: Transfer Decision. Sets input.uc4Go = true only for yes.
2. If input.uc4Go is true: Code: Build ACW, then Say {{context.uc4Cfg.messages.transfer}} (plain Say node), then Code: Mock Transfer, then Hang Up.
3. Else: Resolve Tool Action. The result already went out through actions.output.

Code: Transfer Decision

```
// Code: Transfer Decision. No return statements.
const a = input.aiAgent?.toolArgs || {};
if (!context.shortTermMemory) {
  context.shortTermMemory = {};
}
const u = context.shortTermMemory.uc4;
const cfg = context.uc4Cfg;
const res = (n, say, extra) => Object.assign({ success: true, next_action: n,
say: say || "", guide: "", verbatim: false }, extra || {});
let out = res("apologise_and_retry_once", "");
input.uc4Go = false;

if (a.merchant_answer === "yes") {
  u.transfer.executed = true;
  u.outcome.result = "transferred_mock";
  input.uc4Go = true;
  out = res("terminal", "");
} else if (a.merchant_answer === "unclear" && !u.transfer.unclearOnce) {
  u.transfer.unclearOnce = true;
  out = res("deliver_guided", "", { guide: "Ask once more, in different
words, whether they would like you to connect them with someone who can help
further." });
} else {
  const reason = u.transfer.reason;
```

```
  Object.assign(u.transfer, { declined: true, offered: false, unclearOnce:
false });
  if (reason === "language") {
    out = res("end_call", "", { guide: "exit_type=language_not_supported" });
  } else if (reason === "other_brand") {
    out = res("steer_back", "", { guide: cfg.guided.continueOrder.replace("
{lob}", u.lob) });
  } else {
    out = res("deliver_guided", "", { guide: cfg.guided.declinedPerson });
  }
}

input.uc4Result = out;
// Only the no and unclear branches hand a result back. The yes branch
continues to the mock transfer.
if (!input.uc4Go) {
  actions.output("", out);
}
```

Code: Build ACW (shared by the transfer, safety and end_call paths)

```
// Code: Build ACW. No return statements.
const u = context.shortTermMemory.uc4;
const segSteps = context.uc4Cfg.steps[u.walk.segment] || [];
const lastStep = segSteps[u.walk.stepIndex] ? segSteps[u.walk.stepIndex].id :
"";
context.shortTermMemory.uc4Acw = {
  uc: "UC4",
  lob: u.lob,
  sessionId: u.callId,
  orders: u.orders.map(o => ({ ref: o.ref, ageDays: o.ageDays, threshold:
o.threshold, submitted: o.submitted })),
  journeys: u.outcome.journeys,
  platform: u.walk.platform || u.prefs.platform || "",
  lastStep: lastStep,
  result: u.outcome.result || "",
  transferReason: u.transfer.reason || "",
  classification: u.transfer.executed ? "others" : "",
  language: u.lang,
  languageSwitches: u.language.switches,
  safetySummary: u.safetySummary || ""
};
```

Code: Mock Transfer

```
// Code: Mock Transfer. Stands in for the Voice Gateway Transfer node in this
phase. No return statements.
const acw = context.shortTermMemory.uc4Acw || {};
api.log("info", "UC4 MOCK TRANSFER " + JSON.stringify(acw));
context.shortTermMemory.uc4.walk.status = "handed_over";
context.shortTermMemory.uc4.closingSaid = true;
// Testers check the log line for classification 'others' and the
transferReason.
```

6.13 escalate_safety

The only transfer without a consent question. It interrupts any journey, including a walkthrough in progress, and the dispute flow is not resumed. No guard, so it is always available. Band 2: TC-011, TC-012, TC-034, TC-035.

Tool description (for the LLM): Call this immediately, before anything else, when the merchant reports a real emergency happening now: someone injured or unconscious, an accident that just happened, immediate danger, a threat or harassment, or a live food safety incident. Do not call it for a question about what to do if something happens, a hypothetical, or a past and resolved event. If you genuinely cannot tell, ask one short question first. Never give medical or first aid instructions.

Parameters (JSON schema)

```
{
  "type": "object",
  "properties": {
    "summary": { "type": "string", "description": "One line on what the
merchant reported, for the agent who takes the call." }
  },
  "required": ["summary"],
  "additionalProperties": false
}
```

Flow under the tool: Code: Safety, Code: Build ACW, Say {{context.uc4Cfg.messages.safety}} (tells them to call their local emergency number if anyone is hurt), Code: Mock Transfer, Hang Up.

```
// Code: Safety. No return statements.
const a = input.aiAgent?.toolArgs || {};
if (!context.shortTermMemory) {
  context.shortTermMemory = {};
}
const u = context.shortTermMemory.uc4;
u.transfer.reason = "safety";
u.transfer.executed = true;
u.outcome.result = "safety";
u.walk.status = "handed_over";
u.safetySummary = String(a.summary || "").slice(0, 300);
```

A hypothetical safety question ("who should I call if my rider has an accident?") is not this tool. It goes to get_approved_answer with topic out_of_scope, which offers a person as a question.

6.14 end_call

Plays the exit message that matches how the call ended, then hangs up. The wording comes from config, never from the LLM. Band 1: closing_a. Band 2: the other exit types.

Tool description (for the LLM): Call this to end the call. closing_a: you asked whether there is anything else and the merchant said no, or said thanks or bye after that question. disengaged: the merchant says they must leave now while work is still open (I'm busy, gotta go, nak pergi dah). could_not_resolve: the merchant declined every remaining option and wants to end. language_not_supported: transfer_to_live_agent told you to. Never call it on a bare goodbye before you have asked if there is anything else. Never call it when the merchant says the order is the other brand.

Parameters (JSON schema)

```
{
  "type": "object",
  "properties": {
    "exit_type": { "type": "string", "enum": ["closing_a", "disengaged",
"could_not_resolve", "language_not_supported"] }
  },
  "required": ["exit_type"],
  "additionalProperties": false
}
```

exit_type	Message (config key)	Band
closing_a	messages.exitClosingA: "Thank you for calling Grab. If there's anything else you need help with, please contact us again. Have a nice day."	1
disengaged	messages.exitDisengaged	2
could_not_resolve	messages.exitCouldNotResolve	2
language_not_supported	messages.exitLanguage	2

Flow under the tool: Code: End Call, Code: Build ACW, Say {{input.uc4ExitMsg}}, Hang Up. Once closingSaid is true the guards hide the other tools, so nothing more is spoken.

```
// Code: End Call. No return statements.
const a = input.aiAgent?.toolArgs || {};
if (!context.shortTermMemory) {
  context.shortTermMemory = {};
}
const u = context.shortTermMemory.uc4;
const m = context.uc4Cfg.messages;
const map = {
  closing_a: m.exitClosingA,
  disengaged: m.exitDisengaged,
  could_not_resolve: m.exitCouldNotResolve,
  language_not_supported: m.exitLanguage
};
const exitType = map[a.exit_type] ? a.exit_type : "closing_a";
input.uc4ExitMsg = map[exitType];
if (!u.outcome.result) {
  u.outcome.result = exitType;
}
u.closingSaid = true;
```

6.15 AI Agent configuration and job instructions

Parameter	Value
AI Agent	Grab AI Assistant (introduces itself as Gebi in Bahasa Melayu)

Parameter	Value
Job	UC4 Cancelled Order Compensation
LLM resource	NiCE tenant standard, low latency model for voice (OI-10)
Temperature	0.2
Tool choice	auto
Max tokens	400
Timeout	8000 ms, fallback Say: "Sorry, give me a moment."
Memory	Short term memory on, full call transcript. Do not inject context.shortTermMemory.uc4 into the prompt.
Stream output	On, stop tokens . ! ?
Knowledge at node level	None (E-02)
Safety flags	All four enabled
Tools	The eight tools in 6.1, each with its guard in the Condition field

Job instructions (paste into the AI Agent node)

```
ROLE
You are Grab's AI assistant (Gebi in Bahasa Melayu) on a voice call with a
Grab merchant partner in
Malaysia. This call is about {{context.shortTermMemory.uc4.lob}} orders only.
The merchant was verified
before reaching you. You help with one thing: compensation for an order
cancelled after it was prepared.

LANGUAGE
Merchants speak English, Bahasa Melayu or Manglish (English and Malay mixed
in one sentence).
Understand all three. Reply in the current conversation language:
{{context.shortTermMemory.uc4.langName}}.
- Manglish is not a request to switch. Keep the current language; a light mix
of common Malay words
  (lah, boleh, ok, dah) in your reply is fine when the merchant talks that
```

```
way.
- If the merchant asks for the other language, or speaks mainly in it for a
full turn, call
  get_approved_answer with topic language_request and that language.
- If they ask for a language other than English or Bahasa Melayu:
get_approved_answer, topic
  language_request, language other.
- say with verbatim true is already in the current language: say it exactly.
- say with verbatim false, and every guide: write your reply in the current
language.
- Keep screen names in English in both languages (Orders, History, Cancelled,
Request for Compensation,
  Help Centre, Owner, Manager). In Bahasa Melayu, say pampasan for
compensation.
- Common Malay and Manglish answers: ya, boleh, ok lah, dah, sudah = yes or
done; tak, tak nak,
  tak payah, takpe, tak mau, tamau = no; jap, kejap, tunggu = wait; takde,
tak nampak, tak jumpa =
  not there, not visible or not found.

HOW YOU WORK
1. Your tools hold the rules. After every tool call, do what its next_action
says.
   - say filled and verbatim true: say it exactly, word for word.
   - say filled and verbatim false: you may rephrase it naturally.
   - guide filled: write your own short reply that does exactly what guide
says and keeps every
     fact in it. Add no facts of your own.
2. If the merchant asks a direct question, answer it first, then continue the
current step.
3. One question per turn. Short sentences. No markdown, lists or symbols. Say
numbers as words.
4. Never ask for the order ID, phone number, restaurant name, passwords or
codes.
5. Never ask or confirm whether the order is GrabFood or GrabMart. You
already know.
6. Never say refund. Always say compensation (pampasan).
7. Never promise a callback, follow up or outcome.
8. Never say how many days have passed, what the threshold is, or which path
you chose.
9. Never tell the merchant their order is too old, past a window or not
eligible because of time.
10. Only offer to connect the merchant with a person when a tool returns
```

```
offer_transfer, using its words.
    Never call that person a team, department or specialist.
11. Never state a policy, timeline, eligibility rule or screen name that did
not come from a tool.

WHEN TO CALL WHICH TOOL
- Merchant gives or corrects when the order was cancelled, mentions a
cancelled order without a date,
  or raises another cancelled order: resolve_order_age.
- Merchant asks a follow up after the timeline, asks about evidence, has no
photo, says the order was
  cancelled before it was prepared, raises something outside cancelled order
compensation, says payout
  without a cancelled order, asks for or moves to another language, is
unhappy with you, asks for a
  person, or says the order is the other brand: get_approved_answer with the
matching topic.
- Merchant agrees to be shown or guided, or asks to submit a request:
set_guidance_preference until
  it returns start_walkthrough, then start_walkthrough.
- Every merchant reply during a walkthrough: advance_walkthrough with the
outcome that fits their words.
- Any other cancelled order compensation question: say you don't have that
detail on hand, then ask
  if there is anything else.
- Merchant answers an offer of a person: transfer_to_live_agent.
- A real, active emergency happening now: escalate_safety immediately. Not
for questions about what
  to do if something happens.
- Merchant is finished after you asked if there is anything else, or must
leave now: end_call.

OUT OF SCOPE
Delivery or rider status, account or bank changes, menus, onboarding, app
faults, deductions such as
Eater Compensation, and payouts not tied to a cancelled order.

WAITING
If the merchant says wait, they are checking, logging in, or their device is
slow, call
advance_walkthrough with needs_time during a walkthrough, otherwise
acknowledge in a few words and stop.
Do not offer a person or end the call.
```

6.16 Build checklist

- [ ] Tool name, description and JSON schema copied exactly from this section.
- [ ] Condition field set to the guard in 6.1.
- [ ] Code node guards shortTermMemory, reads input.aiAgent.toolArgs, and writes only to context.shortTermMemory.uc4.
- [ ] No return keyword anywhere in a Code node.
- [ ] Returning tools end with actions.output("", out), followed by Resolve Tool Action.
- [ ] Tier 1 text comes from fixed, walkFixed, steps or messages with verbatim true. Tier 2 text goes out as guide.
- [ ] Terminal paths use plain Say nodes from config, then the mock transfer or Hang Up.
- [ ] No null written to context. Every new field has a default in Code: Initiate.
- [ ] Each tool unit tested in the Interaction Panel with context.shortTermMemory.uc4 preset, then the benchmark cases it unlocks run as text (Section 8).

7. Build Sequence

The build runs in five slices. Slices 1 to 4 make the 15 benchmark cases pass end to end, in an order where each slice can be tested as soon as it lands. Slice 5 adds the Band 2 capabilities.

Slice	Delivers	Lead	Benchmark cases unlocked	Done when
1. Foundation	WebRTC client and test page, Code: Config, language packs (English), phase overrides, Code: Initiate, Once and greeting, English Set Session Config with Malay as alternate, silence handling, mock transfer, end_call (closing_a), AI Agent instructions v1	Dev A	None yet	A WebRTC call reaches the AI Agent, greets, and closes cleanly; actions.output reaches the agent (OI-19).

Slice	Delivers	Lead	Benchmark cases unlocked	Done when
2. Timeline	resolve_order_age (Band 1), get_approved_answer (further_inquiry)	Dev A	TC-001, TC-002, TC-004, TC-025, TC-026, TC-028	Six cases pass as text.
3. Guided status and submission	Step catalogue, set_guidance_preference, start_walkthrough, advance_walkthrough core (confirmed, status outcomes, History fallback, submitted, yes and no, needs_time)	Dev B	TC-003, TC-005, TC-027, TC-029	Four cases pass as text, one by one and all at once.
4. Exceptions	advance_walkthrough repeat, link_not_found, route 1, scroll and role path; get_approved_answer out_of_scope; transfer_to_live_agent (mock)	Dev B and Dev A	TC-006, TC-007, TC-008, TC-009, TC-010	All 15 benchmark cases pass as text. Voice runs start.
5. Band 2	escalate_safety; full stuck ladder; portal routes 2 and 3 with soft close; view order journey; unclear date, correction, second order; dissatisfaction and person request; language_request; Malay Set Session Config and Malay pack; GrabMart mirrors	Both	None (Band 2 cases)	Band 2 cases tested and defects logged.

Order of work inside the phase. Slices 2 and 3 run in parallel (Dev A and Dev B). Slice 4 starts as soon as slice 3's confirmed path works. Slice 5 starts only after the 15 benchmark cases pass as text, and its work stops if a benchmark regression appears.

8. Testing and Traceability

The phase is accepted on Band 1 only. Band 2 is tested after Band 1 passes as text, and its defects do not block the benchmark run.

Stage	What runs	Pass rule
Unit	Each Band 1 tool feature against its decision table in Section 6	Expected next_action and state
Slice text runs	The cases each slice unlocks (Section 7), as text in the Interaction Panel	Same objective as the SRC-1 transcript
Benchmark voice runs	All 15 cases over WebRTC, 3 runs each, with some turns spoken in Manglish	All 3 runs pass
Band 2 runs	Band 2 cases as text, then a voice sample; Malay conversation runs once the Malay pack is loaded	Defects logged by capability

8.1 Band 1 traceability (acceptance)

TC	LOB	Scenario	Tools and features	Requirements
TC-001	Food	Within, timeline, close	resolve_order_age, end_call	R-01, R-02, R-15
TC-002	Food	Within, follow up, guidance declined	resolve_order_age, get_approved_answer (further_inquiry), end_call	R-01, R-04, R-15
TC-003	Food	Beyond, status and dispute, app, one by one	set_guidance_preference, start_walkthrough (status_then_submit), advance_walkthrough	R-05, R-06, R-07, R-10, R-16
TC-004	Food	Beyond, timeline only	resolve_order_age, end_call	R-03
TC-005	Food	Beyond, status and dispute, app, all at once	as TC-003, all at once chunks	R-05, R-06, R-07, R-10

TC	LOB	Scenario	Tools and features	Requirements
TC-006	Food	App form not visible, role path	advance_walkthrough (scroll, role reminder, retry yes or no)	R-12
TC-007	Food	Stuck on a step, repeat, recover	advance_walkthrough (failed: repeat; History fallback)	R-08, R-13
TC-008	Food	App link missing, portal fallback	advance_walkthrough (link_not_found, portal_submit route 1)	R-11
TC-009	Food	Portal form not visible, retry fails, transfer	advance_walkthrough (role path), transfer_to_live_agent (mock)	R-12
TC-010	Food	Out of scope, consent, transfer	get_approved_answer (out_of_scope), transfer_to_live_agent (mock)	R-14
TC-025	Mart	Within, timeline, close	resolve_order_age, end_call	R-01, R-02
TC-026	Mart	Within, follow up, guidance declined	as TC-002	R-04
TC-027	Mart	Beyond, status and dispute	start_walkthrough (portal_status, portal_submit), partial compensation line	R-06, R-09, R-16. The transcript checks status in the app; the build follows portal only (OI-03).
TC-028	Mart	Beyond, timeline only	resolve_order_age, end_call	R-03
TC-029	Mart	Beyond, status and dispute, all at once	as TC-027, portal redirect when the app is chosen	R-05, R-06, R-09

8.2 Band 2 traceability (build now, test later)

Capability	Test cases	Tools and features	Expected behaviour in this phase
Safety escalation	TC-011, TC-012, TC-034, TC-035	escalate_safety	Emergency line, then immediate mock transfer, upfront or mid walkthrough
Portal find method ladder	TC-013 to TC-016, TC-036 to TC-039	advance_walkthrough route_not_found	Route 1, then 2, then 3. All three failing ends in a soft close (E-03).
View Cancelled Order	TC-017, TC-040	start_walkthrough view_order, app_view_order, portal_view_order	Order details open, then the offer to submit
Hard stuck, then transfer	TC-018, TC-041	Full stuck ladder	Repeat, repeat, clarify, clarify, then an offer and mock transfer
Unclear date	TC-019, TC-042	resolve_order_age date ask cap	Date asked twice, then the beyond path with no assumption announced
Dissatisfaction, then request for a person	TC-020, TC-043	get_approved_answer dissatisfied, human_request	One real retry, then an offer and mock transfer
Two orders in one call	TC-021, TC-044	resolve_order_age is_new_order	Order B named as separate, its own caps, preferences reused

Capability	Test cases	Tools and features	Expected behaviour in this phase
Date correction	TC-022, TC-045	resolve_order_age is_correction	Silent recalculation and re-route. TC-045 test data issue noted (OI-05).
Malay language request	TC-023, TC-024	get_approved_answer language_request, languageSwitch = ask_then_transfer	TC-023: continues in English. TC-024: offer, then mock transfer.
GrabMart mirrors	TC-030 to TC-033	Same as TC-006, TC-007, TC-009, TC-010	As the GrabFood cases
Bahasa Melayu conversation	Malay run of the 15 benchmark scenarios	Malay pack, Malay Set Session Config, languageSwitch = switch	Same objectives, replies in Bahasa Melayu, screen names in English

9. Open Items for This Edition

IDs follow TDD v0.3 so both documents track the same items. Items that only affect deferred work are listed at the end.

9.1 Needed for Band 1 (benchmark)

ID	Item	Owner	Needed by
OI-24	Written agreement that the WebRTC call and the mock transfer (message plus 'others' logged) are valid for the benchmark, including TC-009 and TC-010	Grab, NiCE	Week 1
OI-19	Confirm the actions.output payload reaches the AI Agent as the tool result	Amansob	Slice 1

ID	Item	Owner	Needed by
OI-23	Confirm the STT recognises English and Malay together well enough for Manglish	NiCE, Amansob	Slice 1
OI-03	TC-027 checks GrabMart status in the app; the build follows portal only	Grab	Before benchmark voice runs
OI-04	Timeline wording: approved and paid in the next payout cycle, versus credited to the bank account	Grab	Slice 2
OI-13	TC-006: the flow path says the retry is declined, the transcript shows a successful retry. Both are supported; confirm which run Grab will test.	Grab	Slice 4
OI-20	Sign off on the Tier 1 fixed wording used in the benchmark paths	Grab	Before benchmark voice runs

9.2 Needed for Band 2

ID	Item	Owner
OI-12	Portal routes all failing: soft close (SRC-4, this edition) or an offer of a person (TC-016 and TC-039 transcripts)	Grab
OI-11	Full stuck ladder shape for TC-018 and TC-041	Grab
OI-09	TC-023 and TC-024 tested in ask_then_transfer mode; production language behaviour once Malay conversation is live	Grab
OI-21	Approval of the Malay content pack	Grab
OI-22	Malay test cases for the Malay conversation run	Grab, Amansob
OI-05	TC-045 test data: "5 days ago" is within threshold for GrabMart	Grab

9.3 Deferred with Band 3

OI-08 and OI-18 (knowledge store and source), OI-01, OI-02 and OI-07 (CXone keys, transfer destination, ACW delivery) do not apply to this phase.